

**VIETNAM NATIONAL UNIVERSITY - HO CHI MINH
CITY**

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

**FACULTY OF COMPUTER SCIENCE AND
ENGINEERING**



**IOT APPLICATION DEVELOPMENT
REPORT**

SEMESTER: 242 ACADEMIC YEAR: 2024 - 2025

Lecturer: Lê Trọng Nhân

Full name	Student ID	Contribution
Nguyễn Huy Tài	2110513	100%
Trần Đình Tính	2252818	100%
Nguyễn Nhật Khôi	2252379	100%
Nguyễn Hoàng Ngọc Tuấn	2252873	100%

Ho Chi Minh City, May 2025

CONTENTS

1	INTRODUCTION	4
1.1	Background and Motivation	4
1.2	Project Overview	5
1.3	Objectives	6
1.4	Significance of the Project	6
1.5	Global Trends and Future Directions	7
2	SYSTEM DESIGN	8
2.1	System Architecture	8
2.2	Hardware Design	9
2.2.1	Sensor Nodes	9
2.2.2	Edge Device	11
2.3	Core IoT Dashboard: Design and Implementation	12
2.3.1	Dashboard Structure and Features	12
2.3.2	Integration and Communication	13
2.3.3	User Experience and Customization	13
2.3.4	Security and Reliability	13
2.4	Methodology of Transmission Data	14
2.4.1	Telemetry	14
2.4.2	Attribute	14
2.4.3	RPC (Remote Procedure Call)	15
2.5	AI-Based Anomaly Detection and Rule Chain	15
2.5.1	Multivariate Model for Anomaly Detection	15
2.5.2	Rule Chain on Core IoT Platform	16
2.5.3	System Workflow Summary	17
2.5.4	Benefits and Considerations	18
2.6	Web Application: Design and Implementation	18
2.6.1	Architecture, Technology Stack, and Security	18
2.6.2	Core Features, User Interface, and API Integration	19
2.6.3	Code Organization, Extensibility, Testing, and Deployment	20

3	IMPLEMENTATION	22
3.1	Device Implementation	22
3.1.1	MCU and Sensors Implementation	22
3.1.1.1	Sensor Integration	22
3.1.1.2	Task-Based Sensor Handling (FreeRTOS)	22
3.1.1.3	Telemetry Logging and Monitoring	24
3.1.1.4	RPC and Local Control	26
3.1.2	Edge Device Implementation – AI-Based Prediction Model for Anomaly Detection	27
3.1.2.1	Overview of Edge Workflow	27
3.1.2.2	Model Design and Training	28
3.1.2.3	Real-Time Inference Pipeline	29
3.1.2.4	MQTT Communication and Remote Control	30
3.1.2.5	Summary of Edge Responsibilities	31
3.2	Web Application Deployment	31
3.2.1	Project Structure	31
3.2.2	Core Components and Pages	32
3.2.2.1	App.tsx – Main Router	32
3.2.2.2	DeviceDetail.tsx – Device Telemetry Page	33
3.2.2.3	Map.tsx – Device Location Visualization	33
3.2.3	Authentication and Configuration	33
3.2.4	Testing and Monitoring	34
3.2.5	Build and Deployment	34
4	DEMONSTRATION	35
4.1	Dashboard Overview	35
4.2	Entity Aliases and Widgets	37
4.2.1	Widget Data Source and Keys	37
4.2.2	Entity Alias Configuration	38
4.3	Device Profile Configuration	39
4.3.1	General Configuration	39
4.3.2	Alarm Rules Configuration	40
4.3.3	Rule Chain Execution	43
4.4	Web Application Integration	44
4.5	Demo Link	46
5	Prototype: Over-the-air Update	47
5.1	The Implementation and Results of OTA	48
5.1.1	Designing the Rule Chain for OTA Updates	48
5.1.2	Flow chart for integrating the ESP32 with the Core IoT SDK	50

5.1.3	Uploading and Assigning Firmware on the Core IoT Platform	51
5.1.3.1	Uploading Firmware in Core IoT Platform	51
5.1.3.1.a	Uploading Firmware in Core IoT Platform	52
5.1.4	OTA Update Results and Verification	53
6	CONCLUSION	54
6.1	Reflections and Learned Knowledge	54
6.2	Project Achievements	54
6.3	Limitations of the Project	55
6.4	Future Extensions and Vision	56
7	REFERENCES	57

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The world is currently undergoing a profound digital revolution that is transforming the way societies function, how businesses operate, and the manner in which individuals engage with technology in their everyday lives. At the core of this revolution is the rise of the Internet of Things (IoT), a paradigm that connects physical devices—ranging from simple household objects to sophisticated industrial machines—to the digital world. These connections allow for the continuous flow of data, automation of tasks, and the creation of smart environments that respond adaptively to changes in real time.

Over the past decade, IoT has rapidly shifted from a futuristic concept to a tangible reality, with billions of devices now interconnected globally. The growing affordability of microcontroller units (MCUs), advances in sensor technologies, and the availability of powerful cloud computing resources have been instrumental in democratizing access to IoT solutions. The convergence of these technologies has fostered an ecosystem where even small teams and individuals can build robust, scalable, and intelligent systems capable of addressing complex challenges.

IoT has found widespread application across diverse fields. In smart homes, IoT enables seamless control of lighting, heating, cooling, and security, enhancing comfort, safety, and energy efficiency. In agriculture, IoT-driven precision farming techniques help farmers monitor soil moisture, optimize irrigation, and manage crops more efficiently, contributing to increased productivity and sustainability. In healthcare, IoT devices facilitate continuous patient monitoring, support early diagnosis through data analytics, and enable remote healthcare delivery—redefining patient care and improving health outcomes. The manufacturing sector benefits from IoT through predictive maintenance, process automation, and real-time supply chain management, resulting in reduced downtime and increased operational efficiency.

Despite its transformative potential, the development of practical, scalable IoT solutions remains a complex endeavor. Designers must address challenges related to heterogeneous hardware integration, reliable and secure data acquisition, efficient and robust communication be-

tween devices, scalable data storage and processing, and the creation of intuitive user interfaces. Additionally, as the number of connected devices grows, so too does the importance of cybersecurity and data privacy, making these critical considerations in system design.

A significant trend in contemporary IoT development is the integration of artificial intelligence (AI) and machine learning (ML) capabilities. The fusion of AI and IoT—often referred to as Artificial Intelligence of Things (AIoT)—enables systems to analyze large volumes of sensor data, detect patterns, make predictions, and ***autonomously respond instantaneously to user about the weather changes***. This shift from simple data collection to intelligent decision-making unlocks new possibilities for automation, efficiency, and value creation.

1.2 Project Overview

In response to the aforementioned trends and challenges, this project undertakes the design and implementation of a comprehensive IoT system that leverages both hardware and software advancements. The aim is to deliver a smart, adaptable, and scalable solution for real-time monitoring, control, and analytics, suitable for deployment in various real-world scenarios.

The project adopts a modular and layered architecture, with each layer responsible for specific functions and designed to maximize system flexibility and ease of maintenance. The key components of the system include:

- **Device Layer (MCU + Sensors):** The foundational layer comprises microcontroller units connected to a suite of environmental sensors, such as temperature and humidity sensors. This layer is responsible for gathering real-world data, preprocessing it when appropriate, and securely transmitting it for centralized analysis. The system supports straightforward integration of additional sensors and over-the-air (OTA) firmware updates, ensuring that the platform remains adaptable and future-proof.
- **Edge Device:** Situated between the device and higher-level system layers, the edge device aggregates sensor data and performs local computations, such as filtering, data compression, or preliminary analysis. By offloading certain tasks from the cloud, the edge device reduces network latency and bandwidth usage, contributing to a more responsive and efficient system.
- **Dashboard and Visualization:** To empower users with actionable insights, the system features web-based dashboards that present real-time and historical data in a visually engaging manner. Users can monitor device status, configure profiles, customize their dashboards, and receive critical notifications. The dashboards are designed to be intuitive, enabling users of varying technical backgrounds to interact effectively with the system.
- **Artificial Intelligence Module:** Building upon the data collected, AI modules are integrated to perform advanced analytics, such as anomaly detection, predictive maintenance,

and trend analysis. These capabilities not only enhance the system's intelligence but also automate responses to abnormal conditions, thereby improving reliability and safety.

- **Web Application:** Recognizing the importance of accessibility and real-time interaction, the system includes a comprehensive web application. This web-based platform enables users to monitor and control connected devices remotely, visualize real-time and historical data, configure system parameters, and receive timely notifications through any modern web browser. The responsive design ensures seamless user experience across desktops, laptops, tablets, and smartphones, allowing users to interact with the system anywhere and anytime without the need for installing dedicated software.

The system's design prioritizes modularity and scalability, allowing it to be adapted to different application domains with minimal modifications. This flexibility is achieved by abstracting hardware interfaces, adopting standardized communication protocols, and implementing configurable software modules.

1.3 Objectives

The overarching objectives of this project are:

1. To architect and implement a robust IoT system based on modular hardware and software components, supporting seamless integration and extensibility.
2. To establish reliable mechanisms for data acquisition, edge and cloud-based data processing, and secure transmission between system components.
3. To design and develop user-friendly dashboards and mobile applications for real-time monitoring, data visualization, system configuration, and alert management.
4. To integrate AI and machine learning capabilities for intelligent data analytics, early anomaly detection, and autonomous response strategies.
5. To ensure the system's scalability, maintainability, and adaptability to various deployment scenarios, ranging from small-scale experiments to large-scale, real-world applications.
6. To rigorously test the system through multiple deployment scenarios, evaluating its performance, reliability, and user experience, while identifying opportunities for further optimization and future research.

1.4 Significance of the Project

The significance of this project lies in its comprehensive approach to tackling contemporary challenges in IoT system design and deployment. By seamlessly integrating sensing, edge com-

puting, cloud-based analytics, and user-facing applications, the project exemplifies best practices in modern IoT development. The ability to support over-the-air updates, additional sensor integration, and customizable user interfaces underscores the system's adaptability to evolving requirements.

In addition, the project emphasizes the critical role of user-centric design. By focusing on accessibility and customization, the system caters to a broad audience, from technical professionals to end-users with limited technical expertise. The provision of both web-based and mobile interfaces reflects the increasing demand for multi-platform accessibility and real-time engagement in today's connected world.

From an academic perspective, the project provides valuable hands-on experience in hardware-software co-design, embedded systems programming, cloud computing, and AI integration. The multidisciplinary nature of the work fosters collaboration among students with different backgrounds and skill sets, encouraging innovation and creative problem solving.

Moreover, as IoT and AI technologies continue to converge, projects like this serve as a reference model for future research, education, and commercial deployments. The lessons learned and methodologies developed here contribute to the broader knowledge base, inspiring continued progress in the field of smart systems and intelligent automation.

1.5 Global Trends and Future Directions

IoT and AI are widely recognized as foundational technologies of the Fourth Industrial Revolution (Industry 4.0), driving transformation in manufacturing, urban management, environmental sustainability, and beyond. According to recent industry reports, the number of connected devices worldwide is projected to reach tens of billions in the coming years, generating unprecedented volumes of data that will fuel the development of new services and business models.

The future of IoT will likely be shaped by advances in areas such as edge intelligence, energy-efficient device design, enhanced security protocols, and greater interoperability among heterogeneous systems. Additionally, the integration of technologies like 5G connectivity, blockchain, and federated learning is expected to further expand the capabilities and reach of IoT solutions.

This project not only responds to current technological demands but also anticipates future trends by building a platform that is flexible, intelligent, and scalable. The insights gained through this work can inform further innovation in smart cities, digital healthcare, sustainable agriculture, industrial automation, and other emerging domains.

CHAPTER 2

SYSTEM DESIGN

2.1 System Architecture

The system architecture is designed with a modular, multi-layer approach, as depicted in Figure 2.1. This architecture ensures efficient data acquisition, processing, transmission, and visualization, while maintaining flexibility for future upgrades and integration.

At the base of the architecture are the **sensor components**, including a DHT20 (temperature and humidity sensor) and a light sensor. These sensors are connected to a **microcontroller unit (MCU)**, specifically an ESP32-based board, which collects real-time environmental data via serial communication.

The MCU transmits the collected sensor data to the **edge device**, typically a local computer, using serial connection. The edge device serves as an intermediate processing unit—receiving raw data, performing local processing or filtering, and preparing the data for upload.

The edge device then communicates with the **Core IoT cloud server** using the MQTT protocol. This cloud server acts as the central data hub, receiving sensor and AI-processed data for centralized storage, monitoring, and further analysis.

From the Core IoT cloud, the data is distributed to two main interfaces:

- The **Core IoT Dashboard**, which offers an internal management console for system administrators to track, monitor, and configure devices, visualize data trends, and manage deployments.
- A user-facing **Web Application**, which provides real-time visualization, map integration, and interaction with sensor data. Users can view alerts, device status, and other contextual information.

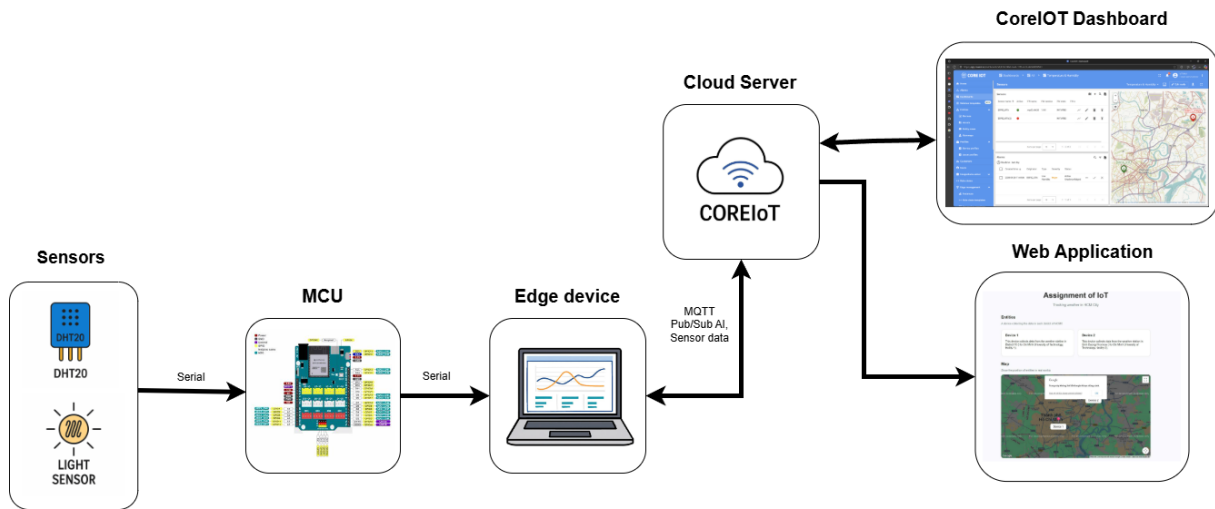


Figure 2.1: Overall System Architecture

2.2 Hardware Design

2.2.1 Sensor Nodes

The sensor nodes are the foundational hardware layer responsible for collecting real-world environmental data. In this project, each sensor node is built around the Yolo UNO microcontroller—a compact and educational development board designed specifically for introductory IoT and STEM applications. The Yolo UNO is based on the ESP32 chipset, featuring a dual-core Tensilica processor operating at up to 240MHz, with integrated WiFi and Bluetooth connectivity, making it ideal for wirelessly connected embedded applications.

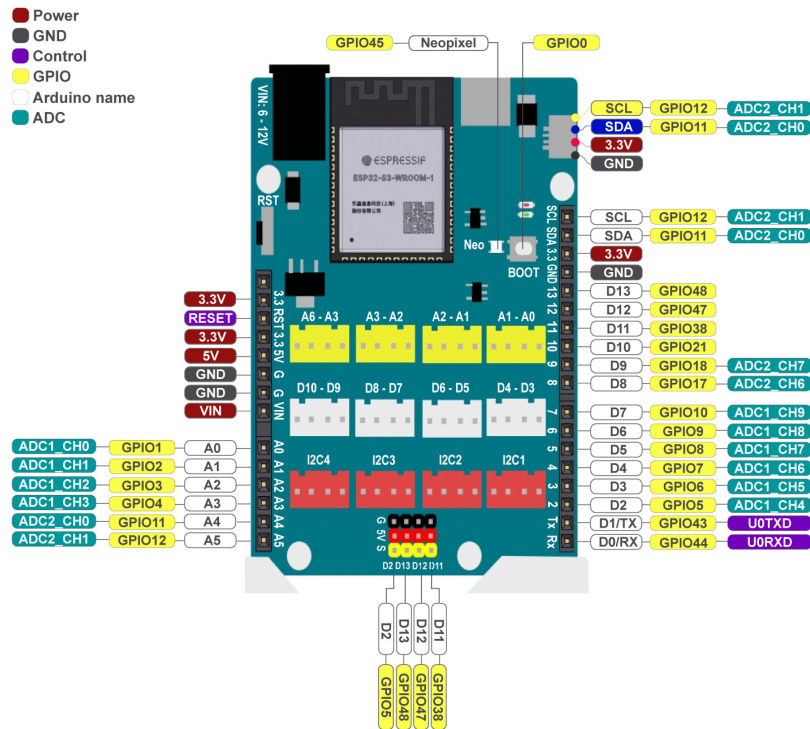


Figure 2.2: Pin diagram of the Yolo UNO board¹

The board provides a user-friendly pin layout and is compatible with a variety of sensors and peripheral modules. In our implementation, each Yolo UNO board is equipped with a DHT20 sensor for measuring temperature and humidity. To enhance environmental monitoring, a light intensity sensor is also included. These sensors are connected to the MCU via digital or analog input pins, typically using the I2C protocol for DHT20 and analog pin readouts for the light sensor.

The firmware programmed on the Yolo UNO is written in MicroPython and leverages event-driven programming techniques. This approach enables the microcontroller to respond efficiently to various sensor events without relying on constant polling, conserving both computational resources and power. The firmware architecture includes modules for serial data transmission, OTA (Over-The-Air) updates, and sensor value handling.

Power is supplied to each sensor node via a micro-USB connection. In stationary lab environments, this setup ensures reliable continuous operation. For potential mobile applications, battery-powered configurations are possible due to the low-power modes supported by the ESP32 chip, which significantly extends operational time under sleep conditions.

A significant feature of the Yolo UNO is its OTA firmware update capability. This feature enables remote updating of the firmware via commands initiated by the edge device or cloud, eliminating the need for physical access. OTA functionality is embedded in the MicroPython firmware using a custom bootloader and update script. This supports a streamlined update workflow that enhances maintainability and reduces system downtime.

In addition to environmental sensing and OTA capabilities, the Yolo UNO supports addi-

tional modules, such as OLED displays or buzzer modules, that can be integrated depending on project requirements. This flexibility allows students and developers to iterate quickly and expand functionality based on use case.

2.2.2 Edge Device

The edge device serves as an intelligent intermediary between the distributed sensor nodes and the Core IoT server. In this architecture, the edge layer is implemented as a Python-based software application, which is designed to run on a general-purpose computer such as a laptop. This design choice ensures access to sufficient computational resources, a user-friendly development environment, and robust connectivity options. The Python edge application is responsible for several critical functions:

- **Data Aggregation:** The edge device collects serial data from multiple Yolo UNO boards connected via USB serial ports, aggregating the sensor readings for unified processing.
- **Preprocessing and Local Analytics:** The received data is validated, filtered, and structured to remove noise and ensure integrity. Light pre-analysis is performed locally, including anomaly scoring using embedded AI models.
- **Anomaly Detection:** A multivariate anomaly detection model is embedded in the edge software. It performs inference on real-time sensor data, detecting potential outliers or hazardous readings.
- **Data Forwarding:** Cleaned and analyzed data is published to the Core IoT server using the MQTT protocol. This ensures efficient, low-latency data streaming and compatibility with ThingsBoard or other dashboard platforms.
- **Remote Configuration and OTA Dispatching:** The edge device listens for configuration updates or firmware dispatch instructions from the cloud and relays them to the corresponding Yolo UNO nodes.

The communication between edge device and MCU nodes is via USB serial, which offers reliable, low-latency data transfer and simplifies device management in controlled environments. Each Yolo UNO is mapped to a unique serial port identifier, and the edge software dynamically handles port enumeration and reconnection. The modular architecture of the edge software allows for extensibility. Core modules are abstracted for sensor mapping, communication handling, data logging, and AI model integration. Configuration parameters such as COM port names, baud rate, and sensor types are defined in a JSON or environment file for maintainability. This edge-based approach introduces resilience: in the event of temporary cloud disconnection, the edge can buffer data locally and resume forwarding when connectivity is restored. Moreover, by executing initial processing and inference at the edge, the system reduces bandwidth use and

latency, enabling faster local responses. Overall, the Yolo UNO + edge device design delivers a robust, educationally grounded, and highly modular IoT system architecture that supports experimentation, real-world deployment, and future AIoT integration.

2.3 Core IoT Dashboard: Design and Implementation

The Core IoT dashboard, powered by the ThingsBoard platform, serves as the main interface for users to monitor, visualize, and manage all connected IoT devices and data streams within the system. It is designed as a web-based application accessible via any modern browser, providing cross-platform compatibility for desktops, laptops, tablets, and smartphones.

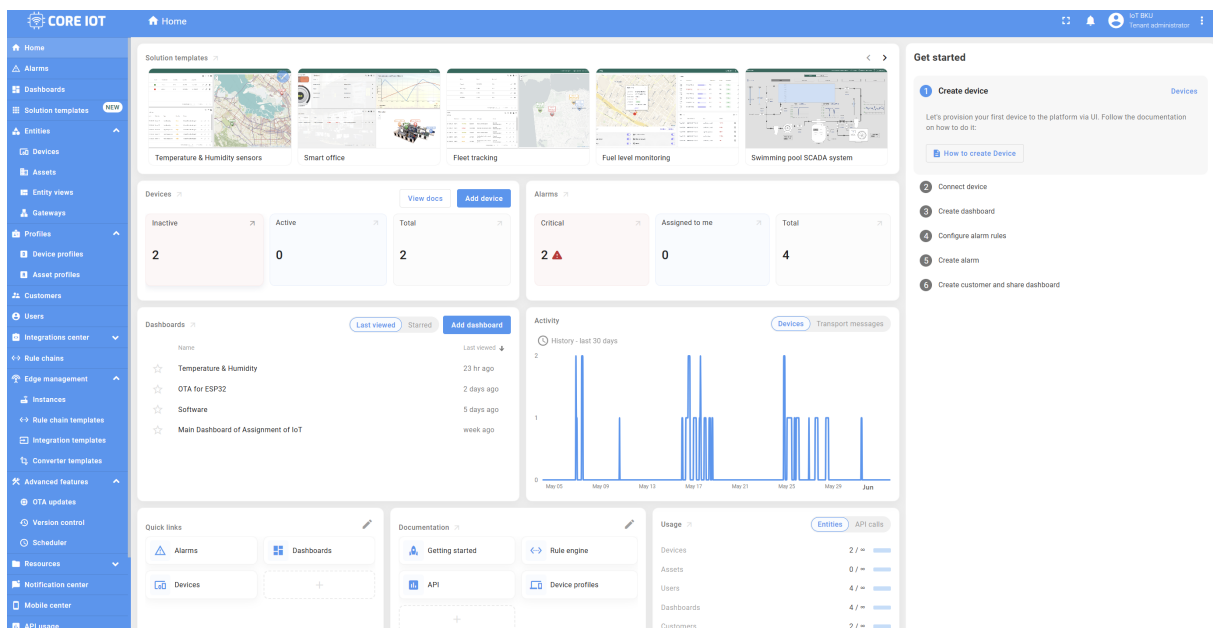


Figure 2.3: Homepage of Core IoT

2.3.1 Dashboard Structure and Features

The dashboard offers a modular and highly configurable environment, enabling users to create and arrange visualization widgets according to their needs. Key functionalities include:

- **Real-Time Telemetry Visualization:** Sensor data—such as temperature, humidity, and other custom metrics—is transmitted from devices (e.g., ESP32 Yolo:Bit) to the platform and visualized in real time using interactive charts, graphs, and gauges. Widgets auto-refresh as new telemetry arrives, ensuring up-to-date status displays.
- **Device and Asset Management:** The platform supports hierarchical organization of devices into assets and groups, allowing users to manage large deployments efficiently. Each device profile includes detailed metadata, connection status, latest telemetry, and can be configured or updated remotely through the dashboard.

- **Alarms and Rule Engine:** The Core IoT dashboard incorporates a rule engine for alarm and event processing. Users can define complex triggers (e.g., threshold breaches, device inactivity, or custom events), which automatically generate alarms, send notifications, or trigger other workflows. Alarm states and histories are clearly visualized within the interface.
- **Remote Device Control and OTA Updates:** Through RPC (Remote Procedure Call) or attribute updates, users can remotely send commands (such as reboot, calibration, or firmware/OTA updates) directly to devices. Status and results of these actions are reflected in real time.
- **Historical Data and Analytics:** Users can query historical telemetry, visualize trends over selectable time ranges, and export datasets for external analysis. The platform also supports integration with external analytics tools or dashboards if required.
- **User and Access Management:** The dashboard supports multiple user roles and permissions (e.g., administrator, tenant, customer, user), ensuring secure, role-based access to devices and data. All actions are logged for audit and security purposes.
- **Custom Widgets and Extensions:** Core IoT allows custom widget development using HTML, JavaScript, and CSS. Users can extend functionality or create bespoke data visualizations tailored to specific requirements.

2.3.2 Integration and Communication

The dashboard communicates with the backend using secure, scalable protocols such as MQTT, HTTP, or WebSocket, depending on device configuration and use case. RESTful APIs are available for integration with third-party services, automation, or custom applications. The data flow is managed efficiently to support both high-frequency telemetry and reliable device command/control operations.

2.3.3 User Experience and Customization

The interface supports drag-and-drop widget arrangement, dashboard templates, and asset hierarchies. Users can save personalized dashboard layouts, apply themes, and filter data based on locations, device types, or alarm states. This enables both operational staff and technical administrators to quickly access relevant information and perform necessary actions.

2.3.4 Security and Reliability

Robust security mechanisms are built in, including encrypted communications (HTTPS, MQTTs), JWT-based authentication, granular access control, and detailed activity logs. The platform is

designed for high availability and reliability, making it suitable for critical monitoring and control applications.

2.4 Methodology of Transmission Data

This section outlines how data is transmitted between the edge device (Yolo UNO board) and the cloud platform using the MQTT protocol. The transmission is organized into three categories: Telemetry, Attributes, and RPC (Remote Procedure Call).

2.4.1 Telemetry

Telemetry refers to data sent from the edge device to the server. These are typically real-time sensor readings or operational status updates. The cloud platform uses this data for monitoring, visualization, and further processing.

- Direction: Edge Device → Server
- Data type: Real-time measurements or events (e.g., temperature, humidity, motion)
- Use case: Monitoring, data logging, analytics

2.4.2 Attribute

Attributes are key-value pairs that define the properties or configuration of a device. They are divided into three types:

- **Client-side attributes:** Sent from the device to the server, typically representing static or semi-static values such as firmware version or network status.
- **Server-side attributes:** Set by the server and can be read or requested by the device. These may contain configuration parameters or thresholds.
- **Shared attributes:** These are server-side attributes explicitly shared with the device, often used for dynamic configuration.
- Direction:
 - Client-side: Device → Server
 - Server-side/Shared: Server → Device
- Use case: Device configuration, metadata, remote tuning

2.4.3 RPC (Remote Procedure Call)

RPC allows the server to remotely invoke actions on the device. It enables bi-directional communication: the server sends a command, and the device performs an action and optionally returns a response.

- Direction: Server $\leftrightarrow$ Device
- Use case: Remote control and automation
- Example: Toggle an LED, activate a motor, request sensor calibration

2.5 AI-Based Anomaly Detection and Rule Chain

The integration of AI-driven analytics with automated SMS notification significantly enhances the responsiveness and reliability of the Core IoT monitoring system. In this architecture, the edge device is responsible for running a pre-trained multivariate anomaly detection model, utilizing both live sensor data and a pre-collected test set. Upon detecting anomalies, the edge device sends relevant details—including raw sensor readings and anomaly status—to the Core IoT (ThingsBoard) platform, where a rule chain orchestrates the web alerting process.

2.5.1 Multivariate Model for Anomaly Detection

Unlike basic threshold alerting, the edge device leverages a pre-trained multivariate statistical model to capture complex abnormal patterns that may involve combinations of sensor variables, such as temperature and humidity. The model was initially trained offline using representative historical data and a designated test set. No additional training is performed during operation; the edge device only performs inference.

Data Flow and Preprocessing: Sensor nodes periodically send their temperature and humidity data to the edge device over a serial connection. The edge aggregates these readings, formats them into feature vectors consistent with those used during model training, and preprocesses the data (handling any missing or noisy values, normalization, etc.).

Inference with Pre-Trained Model: At each time step, the edge device uses the current readings—together with the test set features—to perform real-time anomaly detection. The model calculates an anomaly score (e.g., Mahalanobis distance for a multivariate Gaussian) for the current sensor data. If the score exceeds the established threshold (determined during model validation), an anomaly is flagged.

Anomaly Reporting and Data Forwarding: Upon detecting an anomaly, the edge device constructs a data packet that includes:

- The raw temperature and humidity readings from all involved sensor nodes
- The corresponding values from the test set (for context and traceability)

- The computed anomaly score
- Timestamp and device identification

This packet is transmitted to the Core IoT platform via MQTT or HTTP for further processing.

2.5.2 Rule Chain on Core IoT Platform

On the Core IoT (ThingsBoard) platform, a dynamic and modular Rule Chain is implemented to handle incoming telemetry and anomaly events from the edge device. The rule chain is structured to route and process messages depending on their type (attributes, telemetry, RPC, etc.), and trigger necessary actions such as notifications, data storage, or remote procedure calls.

- **Input Node:** The entry point of all incoming messages (telemetry, attributes, RPC, etc.) from devices connected to the Core IoT server.
- **Message Type Filter:** Filters messages to distinguish between ‘Post attributes’ and ‘RPC’ or other message types.
- **Entity Type Filter:** Checks if the incoming message originates from an Entity Group. If yes, data is duplicated to the group entity for further collective processing.
- **Device Profile Node:** Forwards the message to the corresponding Device Profile, which may contain default alarms or behavior profiles.
- **Message Type Switch:** This node routes each message to its appropriate handler based on the message type:
 - **Attributes Updated:** Routed to the ESP32_OTA‘ rule chain for handling over-the-air firmware updates.
 - **Post Attributes:** Stored using the ‘Save Attributes‘ node and optionally routed to a ‘send_mail (web notification)’ rule chain.
 - **Post Telemetry:** Stored using the ‘Save Timeseries‘ node. If telemetry indicates an anomaly (e.g., temperature or humidity thresholds exceeded), it can be used to trigger alarms.
 - **RPC Request from Device:** Logged using a ‘Log RPC from Device‘ node.
 - **RPC Request to Device:** Passed to ‘RPC Call Request‘ node to issue control commands to the device.
 - **Other:** Logged using the ‘Log Other‘ node.
- **Web Notification (optional alarm integration):** If an anomaly condition is detected based on telemetry (e.g., using a downstream Rule Chain or script logic in the Device Profile), the system may generate an alarm. A Web Notification rule (configured via

Notification Rules in Core IoT Platform) is triggered to display a pop-up alert on the Dashboard for subscribed users.

Example Message Flow:

1. Edge device sends:

```
1 {  
2   "device": "SensorBox01",  
3   "timestamp": 1717321500000,  
4   "temperature": 34.8,  
5   "humidity": 86.1,  
6   "temperature_predict": 33.9,  
7   "humidity_predict": 87.3,  
8   "light": 690  
9 }
```

2. Rule Chain receives this message and routes it based on message type:
 - ‘temperature’ and ‘humidity’ are treated as raw telemetry.
 - ‘temperature_predict’ and ‘humidity_predict’ are prediction outputs from edge AI processing.
 - ‘light’ is logged as environmental sensor data.
3. All telemetry is stored via ‘Save Timeseries’ node.
4. If anomaly detection is handled at the edge and encoded in telemetry (e.g., abnormal deviation between actual and predicted values), the Rule Chain or Device Profile can trigger an alarm.
5. In such cases, a Web Notification is generated and shown on the Dashboard to alert users.

2.5.3 System Workflow Summary

1. Sensor nodes send environmental data to the edge device.
2. Edge device runs an AI-based anomaly detector and forwards all relevant telemetry to Core IoT.
3. Core IoT Rule Chain processes incoming data, saves it, and checks for anomaly conditions.
4. In case of anomaly, a Web Notification is generated and shown to the user in the Dashboard.
5. RPC and OTA functionality is supported for remote control and firmware updates.

2.5.4 Benefits and Considerations

This architecture allows real-time anomaly detection at the edge while leveraging Core IoT's Rule Engine for scalable, no-code message routing. The use of Web Notifications improves user experience over traditional SMS alerts, enabling direct interaction through the Dashboard. System behavior remains transparent and auditable, while modular rule chains offer flexibility for future extension (e.g., integration with third-party APIs or MQTT brokers).

2.6 Web Application: Design and Implementation

The web application serves as the main entry point for users to interact with the IoT monitoring system. Its design focuses on delivering real-time insights, secure access, and powerful data visualization tools, all while ensuring adaptability and future extensibility. The following section provides an in-depth look at the core design aspects and technical implementation choices.

2.6.1 Architecture, Technology Stack, and Security

The application is built using the React framework, utilizing TypeScript for static typing and enhanced code maintainability. React's component-based model is leveraged to structure the interface into reusable, encapsulated blocks such as navigation bars, data visualizations, device cards, and maps. This modularity supports both rapid development and simplified long-term maintenance.

For client-side routing, the application uses `react-router-dom`. This enables a single-page application (SPA) experience, with smooth navigation between key views—such as the main dashboard, device listings, detailed device analytics, and the interactive map—without full page reloads.

State management is primarily handled via React's built-in state hooks and context, ensuring efficient updates and minimal data redundancy. The application supports both global and local state, for instance, tracking authenticated user info, current device context, and map view preferences.

Styling is implemented using CSS modules and scoped classes, allowing for a clean, maintainable codebase with minimal style conflicts. The UI is built around responsive design principles, using media queries and flexible grid layouts to deliver a consistent experience on desktops, tablets, and mobile devices.

Security is a critical focus of the design. The authentication workflow involves submitting user credentials to the Core IoT platform's secure login API

`https://app.coreiot.io/api/auth/login`

Upon successful login, the platform returns a JWT access token, which is stored securely (typically in memory or HTTP-only cookies to mitigate XSS risks). All subsequent API calls,

such as fetching device data or telemetry, include this token in the Authorization header to enforce session security. The application also includes logic for token refresh and session expiry, prompting the user to re-authenticate as necessary.

Sensitive configuration data, such as API endpoints, JWT secrets, or Google Maps keys, are managed through environment variables or external configuration files, excluded from version control to prevent accidental exposure.

2.6.2 Core Features, User Interface, and API Integration

The web application is designed around several major user flows:

- **Home Page and Entity Overview:** The homepage summarizes the system's objectives and provides an at-a-glance status for all registered devices (entities). Devices are displayed as cards with relevant details, including name, location, last update, and quick links to further details.
- **Device Detail and Data Visualization:** Clicking a device navigates to its dedicated detail page. Here, the application fetches live and historical telemetry (e.g., temperature, humidity) from the Core IoT backend using RESTful API endpoints such as

`/api/plugins/telemetry/DEVICE/entityId/values/timeseries.`

Data is rendered using the `recharts` library for line graphs, scatter plots, and tooltips, with interactive axes, zooming, and custom legends for clarity. Data refreshes on a timer or when the user requests updates. The interface also shows device metadata, health status, and related notifications or alarms.

- **Map Integration and Geospatial Management:** The Map view utilizes Google Maps via `google-map-react`. Each device is displayed as a location marker, optionally clustered for dense deployments. Clicking a marker shows summary info and links to the device detail page. The map supports user-friendly features such as zoom, pan, and location search, improving operational awareness for geographically distributed systems.

API interaction is encapsulated in service functions, abstracting fetch logic and error handling away from UI components. All network errors are handled gracefully, providing clear user messages and supporting retries where appropriate.

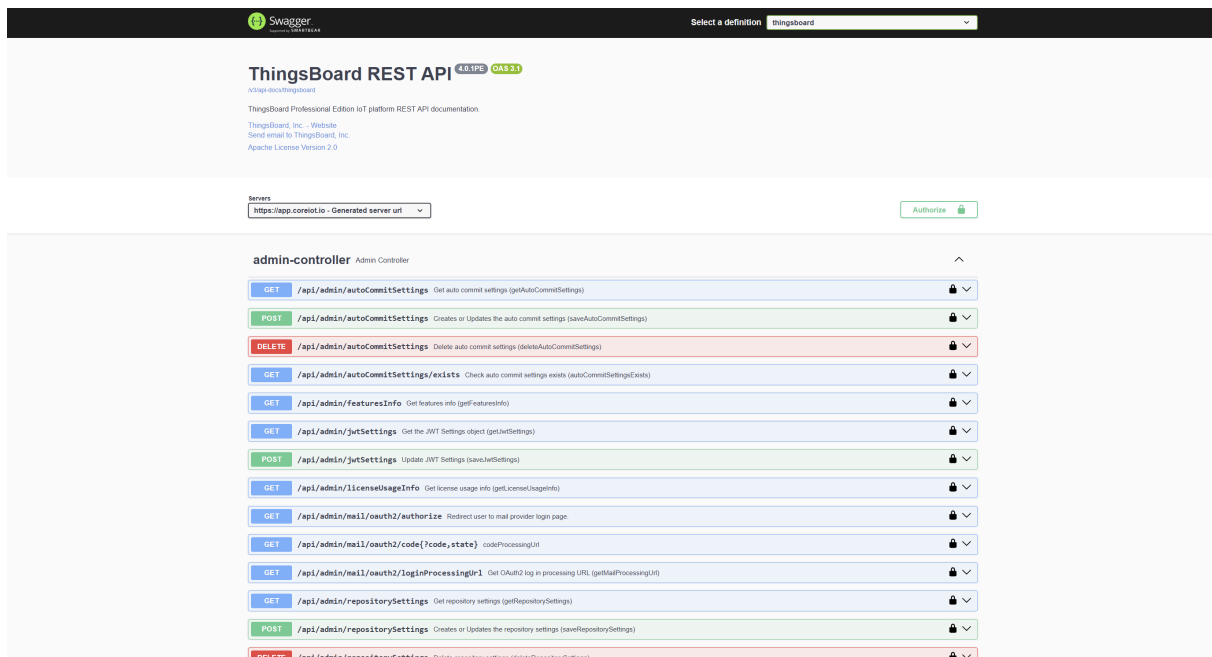


Figure 2.4: Homepage of Core IoT

To facilitate the development and debugging of RESTful API endpoints, the Core IoT platform provides a built-in Swagger UI interface available at:

`https://app.coreiot.io/swagger-ui/`

Swagger UI is an interactive API documentation tool that allows developers to explore, test, and validate the Core IoT REST API directly from the browser. As shown in Figure 2.4, users can interact with various API endpoints such as `/api/admin/autoCommitSettings`, `/api/admin/jwtSettings`, and more. Each endpoint is clearly labeled with its HTTP method (GET, POST, DELETE, etc.), description for authentication, telemetry, attribute, and access control status.

Before accessing secured endpoints, users must first authenticate using the Authorize button, which accepts a JWT access token. This token is typically retrieved from a successful login session and used to validate and authorize API operations.

Swagger UI on Core IoT significantly enhances the development workflow by allowing:

- Quick testing of API functionality without writing client code.
- Immediate feedback on request/response formats and status codes.
- Debugging and validating custom logic or integration points.

2.6.3 Code Organization, Extensibility, Testing, and Deployment

The source code is organized into logical directories, such as:

- **src/**: Core source files, including main app logic, components, hooks, and utilities.

- **components/**: Reusable UI elements such as navigation bars, charts, cards, and forms.
- **pages/**: Higher-level page components for Home, DeviceDetail, and Map views.
- **services/**: API and backend interaction logic, abstracted for maintainability.
- **styles/**: CSS modules and global style files.

Build scripts, configuration files (`package.json`, `tsconfig.json`), and environment files are maintained in the root directory, supporting rapid build and deployment. The application is packaged using the standard React build process, yielding static files suitable for deployment to cloud providers or local web servers.

Testing and code quality are ensured via React's testing utilities and optional TypeScript type checking, reducing runtime errors and simplifying future enhancements. Components are developed with isolation and reusability in mind, facilitating future expansion (such as adding new dashboard widgets, integrating additional device types, or providing more advanced analytics).

Accessibility and user experience are embedded in the design: navigation is intuitive, components use accessible color schemes, and layouts are adaptive. Error boundaries and fallback components are used to handle unexpected issues, further improving robustness.

Routine updates and feature expansion are facilitated by the modular code structure, with clear separation of concerns and adherence to best practices for scalable frontend development.

CHAPTER 3

IMPLEMENTATION

3.1 Device Implementation

3.1.1 MCU and Sensors Implementation

3.1.1.1 Sensor Integration

At the heart of the sensing layer is the ESP32-based microcontroller, selected for its built-in Wi-Fi support, dual-core processor, and FreeRTOS compatibility. The microcontroller interfaces with two key environmental sensors to collect physical data from the environment.

- **DHT20 Sensor:** This sensor provides accurate digital measurements of temperature and relative humidity. It communicates over the I2C protocol, connected via GPIO11 (SDA) and GPIO12 (SCL). The DHT20 is known for low power consumption and high reliability, making it suitable for continuous monitoring applications.
- **Analog Light Sensor:** A photoresistor-based sensor is connected to the analog-capable GPIO1 pin of the ESP32. It outputs an analog voltage proportional to ambient light intensity, which is converted into a digital value using the ESP32's internal ADC (Analog-to-Digital Converter).

3.1.1.2 Task-Based Sensor Handling (FreeRTOS)

Sensor readings are offloaded to individual FreeRTOS tasks to ensure parallel, non-blocking data acquisition. This structure ensures system responsiveness and modular code design.

3.1.1.2.1 Temperature and Humidity Task – taskDHT20 The flowchart below illustrates the cyclic logic used to read and update environmental data every second:

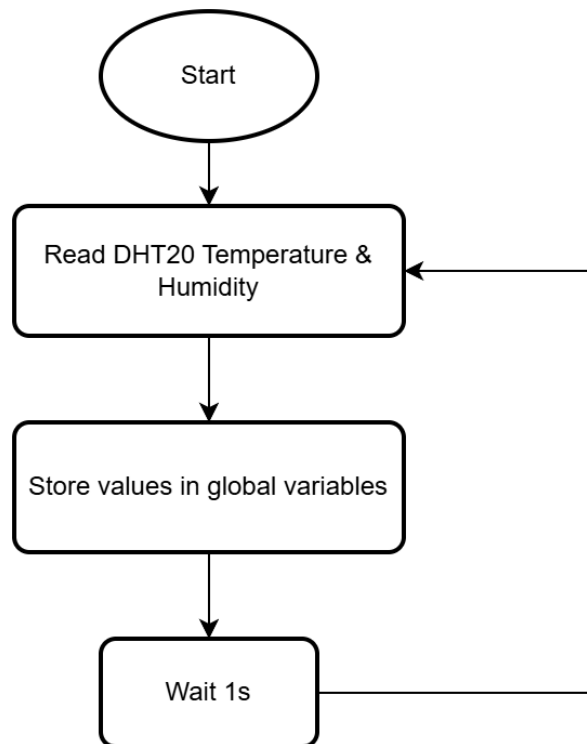


Figure 3.1: Flowchart of taskDHT20: Temperature and humidity reading task.

The DHT20 sensor is polled every second. The temperature and humidity values are stored in global variables for subsequent use in telemetry publishing or alarm evaluation:

```

1 void taskDHT20(void *parameter) {
2     while (true) {
3         dht20.read();
4         temperature = dht20.getTemperature();
5         humidity = dht20.getHumidity();
6         vTaskDelay(1000 / portTICK_PERIOD_MS);
7     }
8 }

```

3.1.1.2.2 Light Sensing Task – taskLight The light sensor task’s logic is described in the following flowchart:

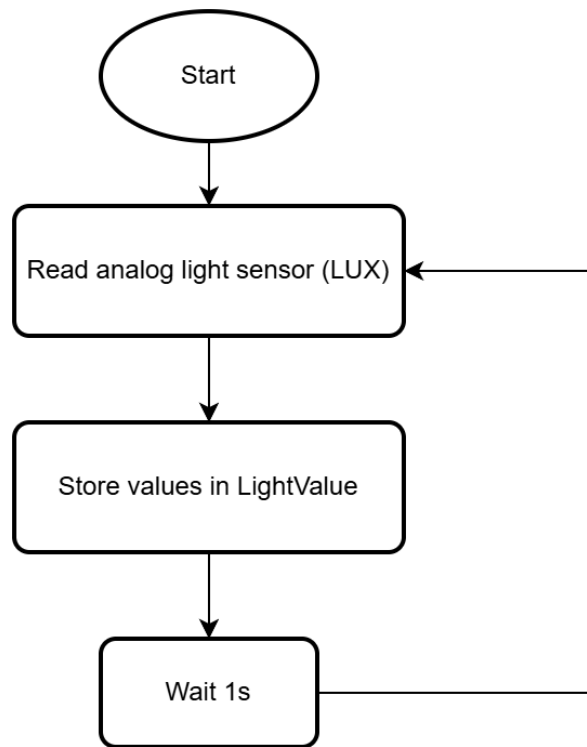


Figure 3.2: Flowchart of taskLight: Light intensity reading task.

Ambient light intensity is read from the analog input pin and stored in a global variable. This value is later used for data visualization, threshold-based alarms, or decision-making:

```

1 void taskLight(void *parameter) {
2     while (1) {
3         LightValue = analogRead(LUX);
4         vTaskDelay(1000 / portTICK_PERIOD_MS);
5     }
6 }

```

3.1.1.3 Telemetry Logging and Monitoring

To facilitate real-time system diagnostics and data validation, a dedicated FreeRTOS task named taskSerial is implemented. This task continuously outputs sensor values — including temperature, humidity, and ambient light — to the serial monitor. This stream of structured telemetry data allows developers to verify sensor functionality, monitor firmware stability, and cross-check transmitted data against the Core IoT backend.

The format of the printed string is prefixed with the keyword DATA:, followed by the temperature, humidity, and light sensor values, separated by commas. This consistent format makes it easy to parse the output with serial monitoring tools or data loggers.

The output example would resemble:

DATA: 28.5, 61.2, 342

DATA: 28.6, 61.0, 347

Such outputs are especially useful during initial hardware bring-up, field debugging, or while verifying calibration of sensors under varying environmental conditions.

3.1.1.3.1 Telemetry Logging and Monitoring Task – taskSerial The following flowchart illustrates the task loop logic, including data acquisition from shared global variables and periodic output to the serial interface:

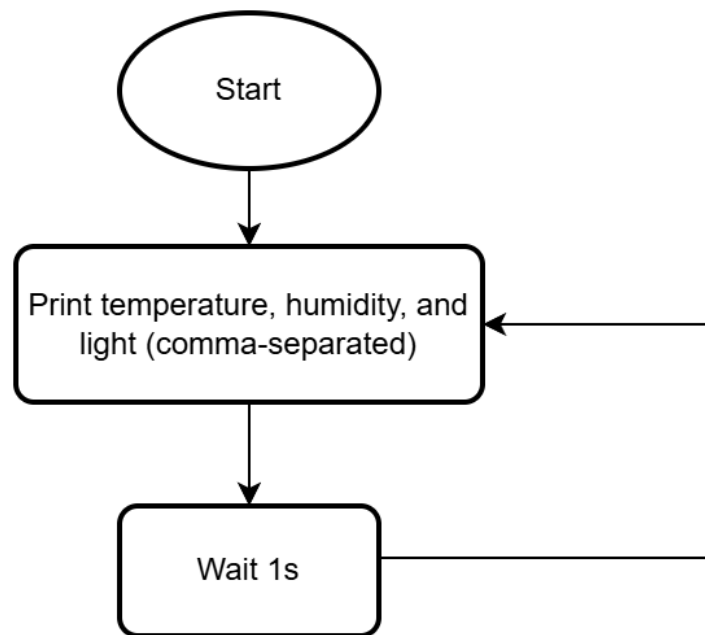


Figure 3.3: Flowchart of taskSerial: Continuous logging of sensor data.

```

1 void taskSerial(void *parameter) {
2     while (1) {
3         Serial.print("DATA:␣");
4         Serial.print(temperature);
5         Serial.print(",␣");
6         Serial.print(humidity);
7         Serial.print(",␣");
8         Serial.println(LightValue);
9         vTaskDelay(1000 / portTICK_PERIOD_MS);
10    }
11 }
  
```

3.1.1.4 RPC and Local Control

3.1.1.3.2 Code Implementation To provide both local and remote device control, the system supports two command interfaces for toggling the indicator LED (GPIO48): one through **serial communication** (USB terminal) and the other via **cloud-based RPC** using MQTT.

3.1.1.4.1 System Overview Flowchart The following flowchart illustrates the decision-making process of the system when handling control commands from either source:

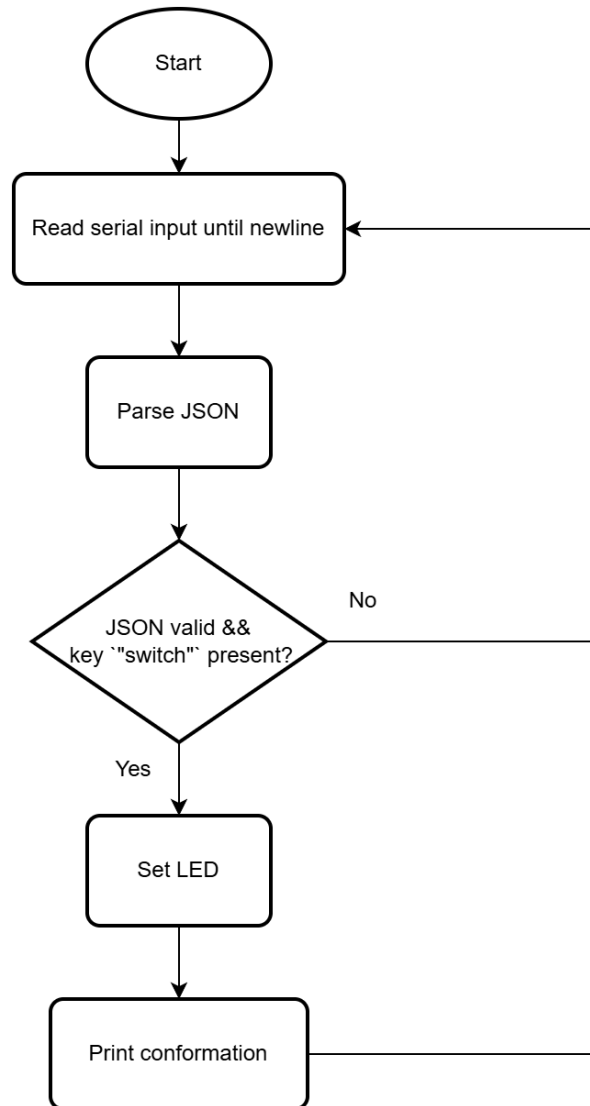


Figure 3.4: Flowchart of taskSerialCommand: Serial-based LED toggle command handler.

3.1.1.4.2 Local Serial Command Task (RPC control) The task continuously listens to incoming serial input. When it receives a full JSON command string (terminated by a newline '\n'), it parses the data using `deserializeJson()` and checks if the "switch" key exists. If valid, it toggles the LED accordingly.

```

1 void taskSerialCommand(void *parameter) {
2     StaticJsonDocument<64> doc;

```

```

3   String input;
4   while (1) {
5       while (Serial.available()) {
6           char c = Serial.read();
7           if (c == '\n') {
8               DeserializationError err = deserializeJson(
9                   doc, input);
10              if (!err && doc.containsKey("switch")) {
11                  bool sw = doc["switch"];
12                  digitalWrite(LED_PIN, sw ? HIGH : LOW);
13                  Serial.printf("Set_LED_by_serial:_%s\n",
14                      sw ? "ON" : "OFF");
15              }
16              input = "";
17          } else {
18              input += c;
19          }
20      }
21      vTaskDelay(10 / portTICK_PERIOD_MS);
22  }

```

Details:

- **Deserialization:** Parses JSON string to extract data.
- **Switch Detection:** Checks if key "switch" is present and valid.
- **LED Output:** Uses digitalWrite to update GPIO48 state.
- **Feedback:** Prints confirmation to serial monitor.

3.1.2 Edge Device Implementation – AI-Based Prediction Model for Anomaly Detection

The edge device in this system plays a crucial role in enabling intelligent, real-time anomaly detection without relying on constant cloud computation. It interacts with a microcontroller (ESP32) to gather raw sensor data over serial, performs local inference using a CNN-based model, and publishes telemetry and prediction results to the cloud via MQTT.

3.1.2.1 Overview of Edge Workflow

The edge device performs a full pipeline locally:

1. Load a pre-trained CNN model for multivariate time-series prediction.
2. Receive real-time sensor data (temperature, humidity, light) via serial from the ESP32.
3. Buffer recent readings to prepare input for inference.
4. Predict next values using AI.
5. Push telemetry (raw and predicted) to Core IoT (via MQTT).
6. Handle remote commands (RPC) from the Core IoT dashboard.

The workflow of the edge device can be visualized by the following flowchart:

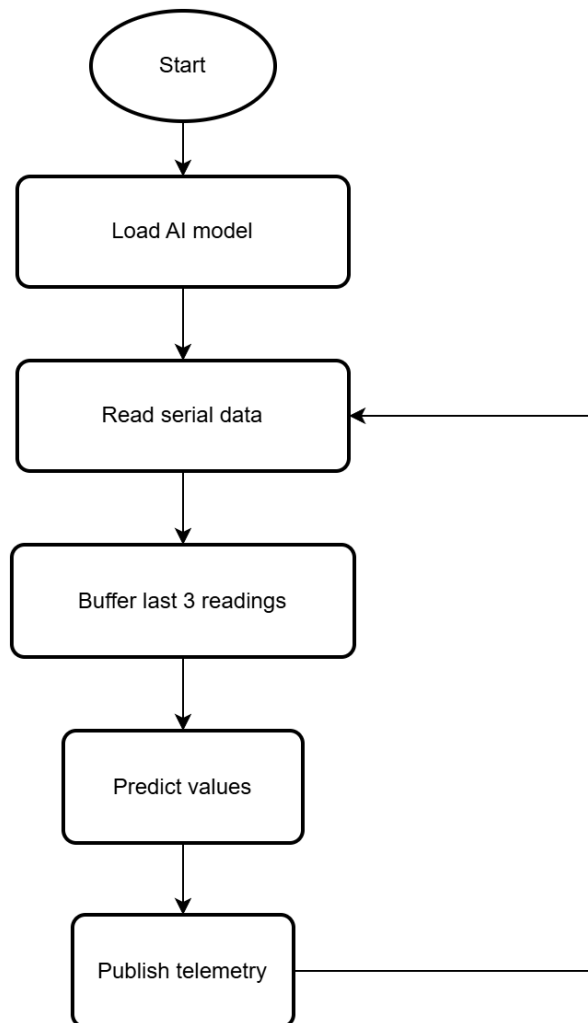


Figure 3.5: Edge Device's Workflow

3.1.2.2 Model Design and Training

Dataset Preparation: The model is trained using environmental logs from `train.csv`, which includes relative humidity and indoor temperature.

Sequence Preparation:

```

1 def split_sequences(sequences, n_steps):
2     X, y = list(), list()
3     for i in range(len(sequences)):
4         end_ix = i + n_steps
5         if end_ix > len(sequences)-1:
6             break
7         seq_x, seq_y = sequences[i:end_ix, :], sequences[
            end_ix, :]
8         X.append(seq_x)
9         y.append(seq_y)
10    return np.array(X), np.array(y)

```

Model Input: Three past time steps of both humidity and temperature are combined:

```

1 humi_seq_train = np.array(train_data['Relative_humidity_room'
    ]).reshape(-1, 1)
2 temp_seq_train = np.array(train_data['Indoor_temperature_room'
    '']).reshape(-1, 1)
3 dataset = np.hstack((humi_seq_train, temp_seq_train))
4 X, y = split_sequences(dataset, n_steps=3)
5 n_features = X.shape[2]

```

CNN Model Training:

```

1 model = Sequential()
2 model.add(Conv1D(filters=64, kernel_size=2, activation='relu',
    , input_shape=(3, n_features)))
3 model.add(MaxPooling1D(pool_size=2))
4 model.add(Flatten())
5 model.add(Dense(50, activation='relu'))
6 model.add(Dense(n_features))
7 model.compile(optimizer='adam', loss='mse')
8 model.fit(X, y, epochs=100, verbose=1)

```

3.1.2.3 Real-Time Inference Pipeline

Step 1: Serial Data Acquisition from MCU

The edge device communicates with the ESP32 through the serial interface to retrieve live sensor values:

```

1 ser = serial.Serial('COM3', 115200, timeout=2)
2
3 line = ser.readline().decode('utf-8').strip()
4 if line.startswith("DATA:"):

```

```

5     data = line.replace("DATA:", "").split(',')
6     temp = float(data[0])
7     humi = float(data[1])
8     light = float(data[2])

```

Step 2: Prediction using Buffered Data

After buffering three samples, the edge performs inference:

```

1 def prediction(array):
2     x_input = np.array(array).reshape((1, 3, n_features))
3     predicted_value = model.predict(x_input, verbose=0)
4     temp_1 = float(predicted_value[0][1])
5     humi_1 = float(predicted_value[0][0])
6     return temp_1, humi_1

```

Step 3: Telemetry Publishing to Core IoT

The device sends real and predicted data to the cloud for storage and dashboard visualization:

```

1 collect_data = {
2     'temperature': temp,
3     'humidity': humi,
4     'temperature_predict': temp_predict,
5     'humidity_predict': humi_predict,
6     'light': light
7 }
8 client.publish('v1/devices/me/telemetry', json.dumps(
    collect_data), 1)

```

3.1.2.4 MQTT Communication and Remote Control

The device connects to Core IoT's MQTT broker using a unique device token:

```

1 client = mqttclient.Client()
2 client.username_pw_set(TOKEN)
3 client.on_connect = on_connect
4 client.on_message = on_message
5 client.connect(BROKER, PORT)
6 client.loop_start()

```

It also handles Remote Procedure Calls (RPC) for control purposes. For example, the Core IOT server can send a 'setSwitch' command to toggle the LED through the ESP32:

```

1 def on_message(client, userdata, message):
2     payload = json.loads(message.payload.decode())

```

```

3     if payload.get("method") == "setSwitch":
4         switch_state = payload.get("params")
5         ser.write((json.dumps({"switch": switch_state}) + "\n"
                        ").encode())

```

3.1.2.5 Summary of Edge Responsibilities

- Local AI-based forecasting of environment metrics.
- Serial data collection from an ESP32 sensor node.
- Detection of anomalies via prediction deviation.
- Realtime telemetry publishing via MQTT.
- Support for cloud-issued control commands via RPC.

This makes the edge device not just a passive data logger, but an active, intelligent participant in the overall IoT ecosystem.

3.2 Web Application Deployment

The web application serves as the primary interface for users to interact with the IoT monitoring system. Built using React with TypeScript, it focuses on displaying real-time telemetry data, visualizing device locations on a map, and ensuring a responsive user experience across platforms.

3.2.1 Project Structure

The project follows a flat folder structure under the `src/` directory, containing core pages and utilities:

```

src/
|- App.tsx           # Root component and routing logic
|- App.test.tsx      # Unit test for the root component
|- DeviceDetail.tsx  # Device telemetry and detail page
|- Map.tsx           # Map view with Google Maps integration
|- index.tsx         # React entry point
|- index.css         # Global styles
|- App.css           # App-wide styles
|- logo.svg          # Branding/logo
|- reportWebVitals.ts # Web performance monitoring
|- setupTests.ts     # Test setup

```

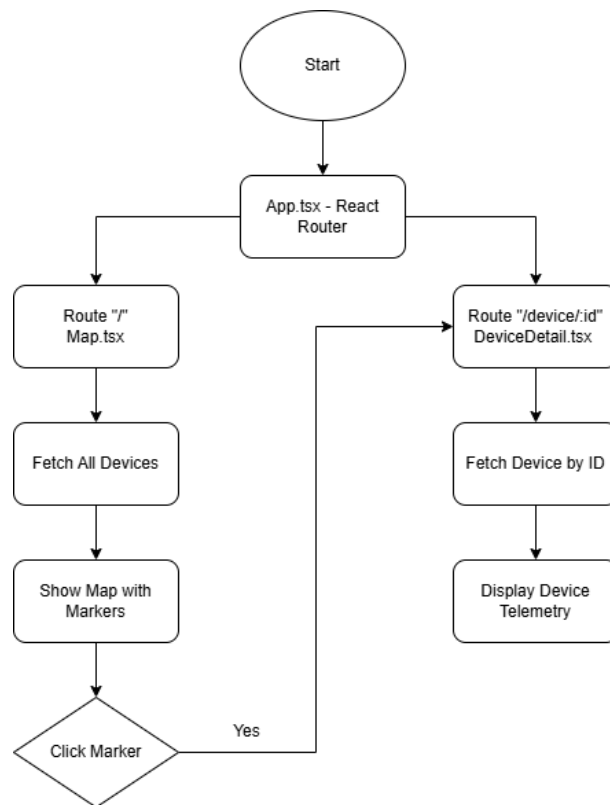



Figure 3.6: Flowchart of Web application.

3.2.2 Core Components and Pages

3.2.2.1 App.tsx – Main Router

This component defines the main application routes using react-router-dom.

```

1 import { Routes, Route } from 'react-router-dom';
2 import DeviceDetail from './DeviceDetail';
3 import MapView from './Map';
4
5 function App() {
6   return (
7     <Routes>
8       <Route path="/" element={<MapView />} />
9       <Route path="/device/:id" element={<DeviceDetail />} />
10    </Routes>
11  );
12 }
13 export default App;

```

3.2.2.2 DeviceDetail.tsx – Device Telemetry Page

This component fetches telemetry data (e.g., temperature, humidity) and visualizes it.

```

1 useEffect(() => {
2   fetch(`/api/plugins/telemetry/DEVICE/${id}/values/
      timeseries`)
3     .then(res => res.json())
4     .then(data => setTelemetry(data));
5 }, [id]);

```

3.2.2.3 Map.tsx – Device Location Visualization

This component renders an interactive map using Google Maps with device markers.

```

1 <GoogleMapReact
2   bootstrapURLKeys={{ key: process.env.
      REACT_APP_GOOGLE_MAP_KEY! }}
3   defaultCenter={center}
4   defaultZoom={10}
5 >
6   {devices.map(device => (
7     <Marker key={device.id} lat={device.lat} lng={device.lng}
8       />
9   ))}
10 </GoogleMapReact>

```

3.2.3 Authentication and Configuration

Authentication is handled using a pair of JWT (JSON Web Token) by the CoreIoT platform. Upon logging in with a valid username and password, the client receives:

- **Access Token (JWT)** – a short-lived token used for executing authenticated API calls.
- **Refresh Token** – used to obtain a new access token after the original one expires.

By default, the access token is valid for **2.5 hours**. The expiration durations of both the access and refresh tokens can be customized in CoreIoT system settings using the parameters `JWT_TOKEN_EXPIRATION_TIME` and `JWT_REFRESH_TOKEN_EXPIRATION_TIME`.

In the client application, the access token is gotten by REST API on Swagg UI platform¹ and included in the headers of all API requests:

¹Swagg UI is described in section 2.6.2

```
1 headers: {  
2   'Authorization': `Bearer ${storedJwtToken}`  
3 }
```

In Core IoT, JWT serves as an access token to securely authenticate and authorize users or devices interacting with the REST API. After logging in, a JWT is issued and must be included in the Authorization header for API requests to get the data in attributes, telemetry, device firmware, etc.

3.2.4 Testing and Monitoring

`App.test.tsx` provides a basic unit test for rendering the main application, while `reportWebVitals.ts` is included to enable optional performance tracking using modern web metrics.

3.2.5 Build and Deployment

The application is built using:

```
1 npm install  
2  
3 npm install recharts  
4  
5 npm install --save-dev @types/google-map-react  
6  
7 npm start
```

This generates a static site ready for deployment to platforms like Vercel, GitHub Pages, or traditional servers using Nginx.

CHAPTER 4

DEMONSTRATION

4.1 Dashboard Overview

The Core IoT platform serves as a centralized control and monitoring hub, providing a comprehensive overview of all sensor nodes and system activities in real time. The main dashboard combines geographic visualization with live telemetry, allowing users to easily track device location, status, and current environmental measurements. Sensor nodes are plotted on an interactive map, each annotated with current readings (temperature, humidity, and light intensity), and can be selected for further inspection.

In addition to visual placement, the system shows a structured summary of each device's state in a table, including firmware name, firmware version, OTA update state, and error status (if any). The bottom section of the dashboard is dedicated to alarm logs, where each row records events such as high temperature, low humidity, or low light levels. These alarms are color-coded by severity (e.g., Minor, Major, Critical) and allow acknowledgment or clearance directly from the interface.

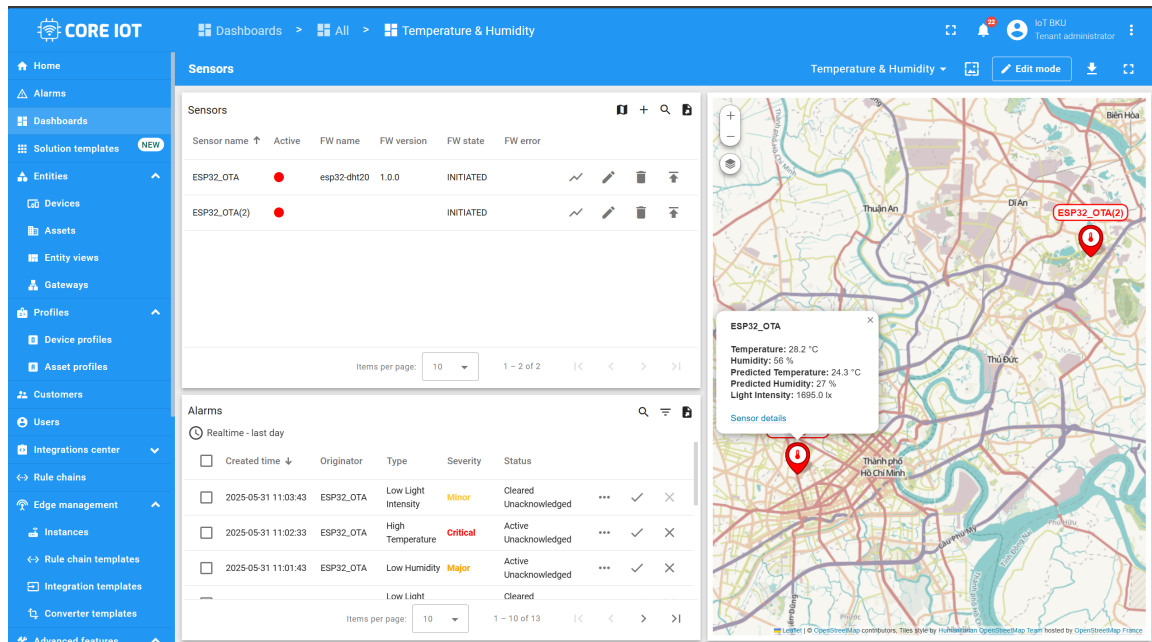


Figure 4.1: Overview of the Core IoT dashboard showing real-time telemetry, device state, location map, and triggered alarms.

When a specific device is selected, the system navigates to its detailed dashboard view. This view aggregates all available data and control elements related to that device:

- **Sensor Settings:** Adjustable threshold sliders for triggering alarms (e.g., high temperature, low humidity, low light).
- **Live Telemetry Charts:** Line graphs plot the last 24 hours of data for each sensor metric (temperature, humidity, and light), allowing users to observe trends and detect anomalies over time.
- **Location Map Panel:** Zoomed-in map view pinpointing the device's exact coordinates.
- **Entity Metadata:** Information such as the firmware version, active status, OTA state, and communication parameters are summarized below the graphs.
- **RPC Control Widget:** A toggle or switch widget allows users to remotely execute commands, such as turning a device's indicator LED on/off or rebooting it.

This dashboard supports modular customization: users can rearrange widgets, add new visualizations, or include third-party integrations. All updates are reflected in real-time using MQTT streams and REST API polling behind the scenes, ensuring that operators are always presented with the most current state of the deployed IoT system.

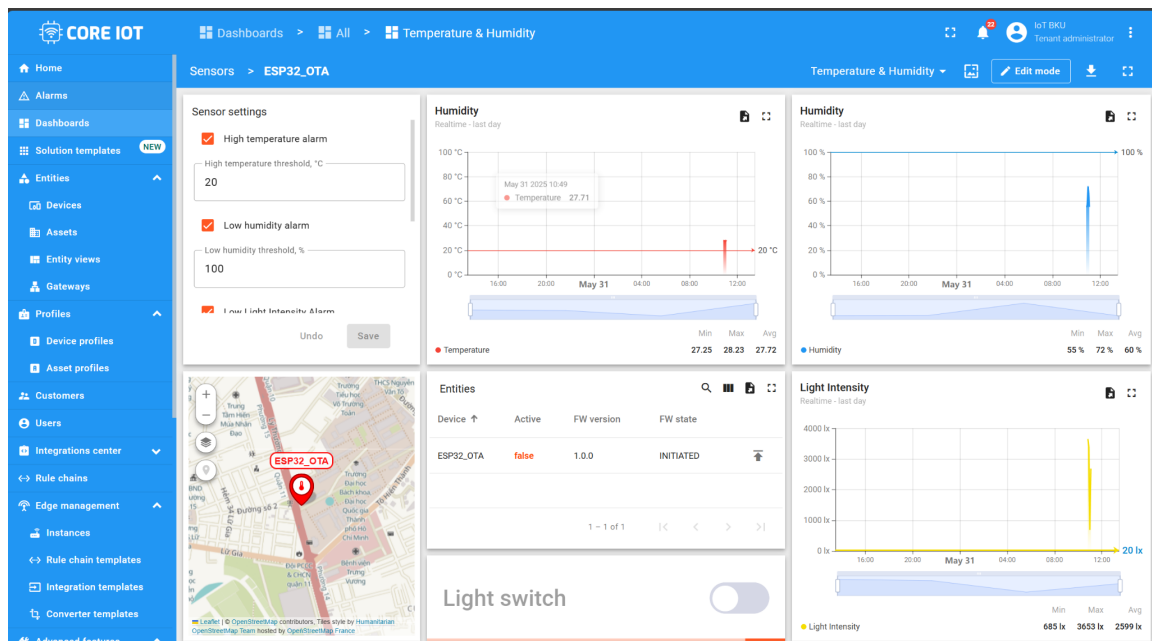


Figure 4.2: Detailed ESP32 sensor dashboard with real-time telemetry, alert thresholds, location, and remote device control.

4.2 Entity Aliases and Widgets

The Core IoT platform utilizes a flexible and modular widget system powered by entity aliases. Widgets are dashboard components that visualize telemetry, display attributes, or allow user interactions (e.g., RPC calls). To achieve reusability and device-independent visualization, each widget links to an **entity alias**—a dynamic reference to one or more devices.

4.2.1 Widget Data Source and Keys

Each widget is associated with an entity alias (e.g., Sensors) and configured to extract relevant data through specific telemetry or attribute keys. In this example, a table widget displays a list of registered sensor nodes along with metadata and firmware states. The keys configured for this entity alias include:

- **Active:** `active` – Indicates whether the device is currently online.
- **FW name:** `current_fw_title` – Identifies the firmware package currently installed.
- **FW version:** `current_fw_version` – Displays the version number of the installed firmware.
- **FW state:** `fw_state` – Shows the firmware upgrade state (e.g., INITIATED).
- **FW error:** `fw_error` – Reports any errors encountered during OTA or configuration updates.

These data keys are continuously updated in real time and presented in a compact tabular format, allowing administrators to quickly assess the status and health of each connected device.

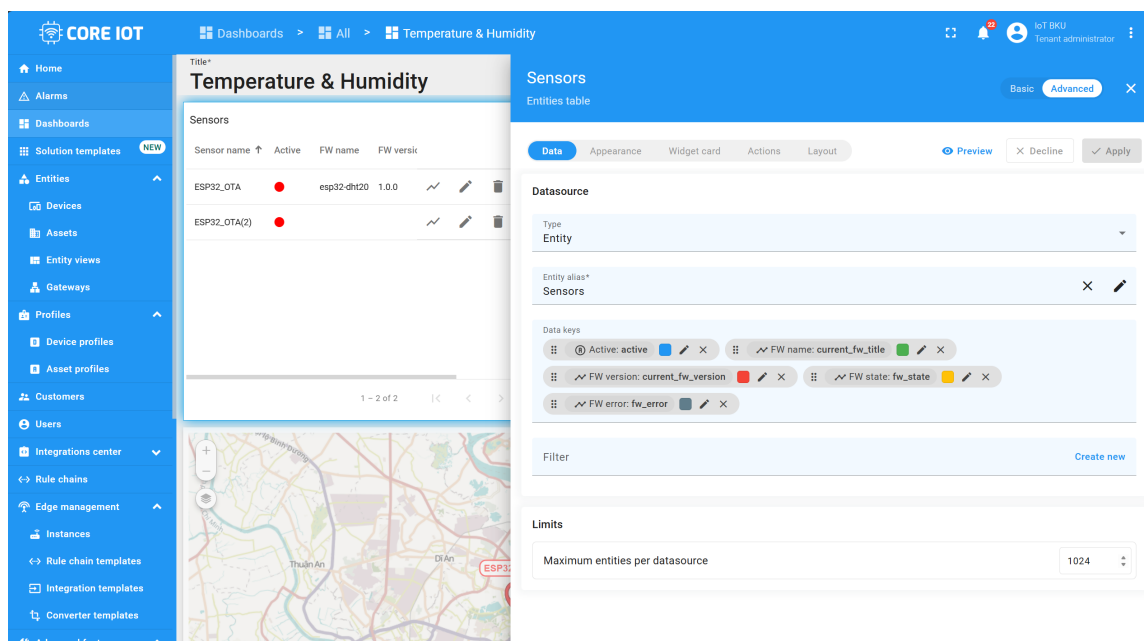


Figure 4.3: Widget configuration with alias-linked data keys displaying firmware status for each device.

This design ensures centralized firmware monitoring, allowing easy identification of devices requiring attention due to outdated versions or failed updates.

4.2.2 Entity Alias Configuration

An *entity alias* in Core IoT represents an abstract reference to one or multiple devices. Rather than binding a widget directly to a fixed device, it dynamically resolves which entity to target based on the dashboard's current state or context.

In the dashboard setup shown below, an alias named *Sensor* is defined. This alias uses:

- **Filter type:** Entity from dashboard state – meaning the selected entity depends on the user's interaction.
- **Resolve as multiple entities:** Enabled – allowing the widget to handle lists or groups of sensors.
- **Default state entity:** a fallback entity that widgets connect to when no user-defined selection is made.

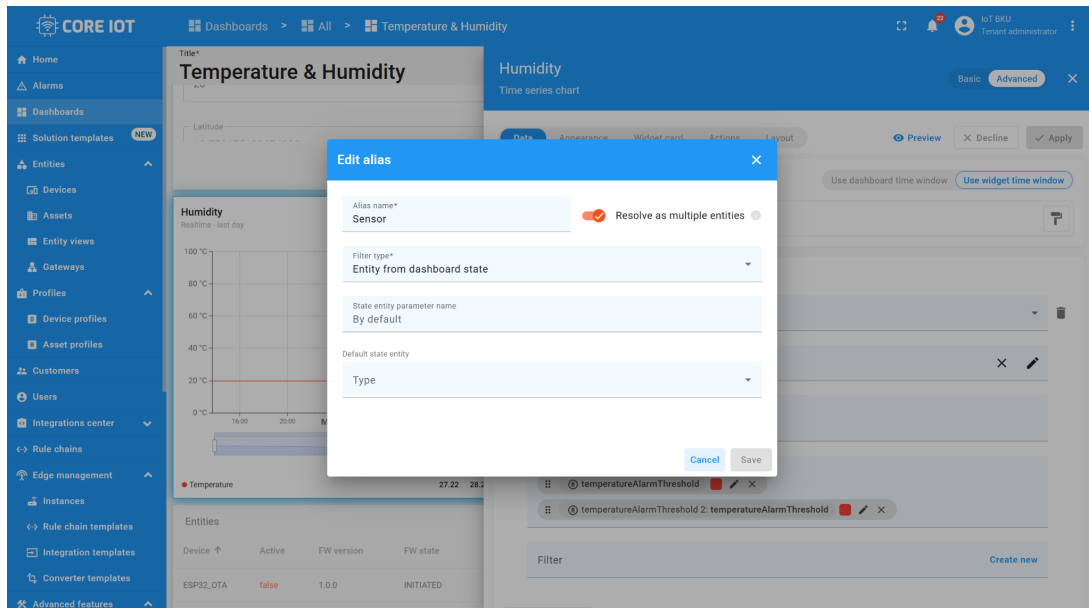


Figure 4.4: Entity alias configured to dynamically resolve based on dashboard state.

This configuration greatly enhances modularity and scalability. For example:

- The same widget can be reused across multiple dashboards or tabs without reconfiguring its data source.
- The user can select any device from a dropdown menu or a list widget, and all linked widgets will automatically update to reflect the selected entity.
- Combined with time filters and interactive controls, dashboards become highly dynamic and user-friendly.

Overall, this entity alias mechanism supports context-aware dashboards, enabling interactive, real-time monitoring across a growing fleet of IoT devices.

4.3 Device Profile Configuration

Device profiles define the expected behavior, data structure, and automation logic for each class of connected device in the Core IoT platform. They are a critical layer for scaling and standardizing device behavior, and serve as the configuration reference for telemetry, alarms, OTA updates, and rule chains.

4.3.1 General Configuration

Each profile includes a unique name, associated transport configuration, and optional fields such as assigned firmware, edge rule chain, and dashboard templates. One of the most important configurations is assigning a default rule chain that governs how telemetry from this profile is processed.

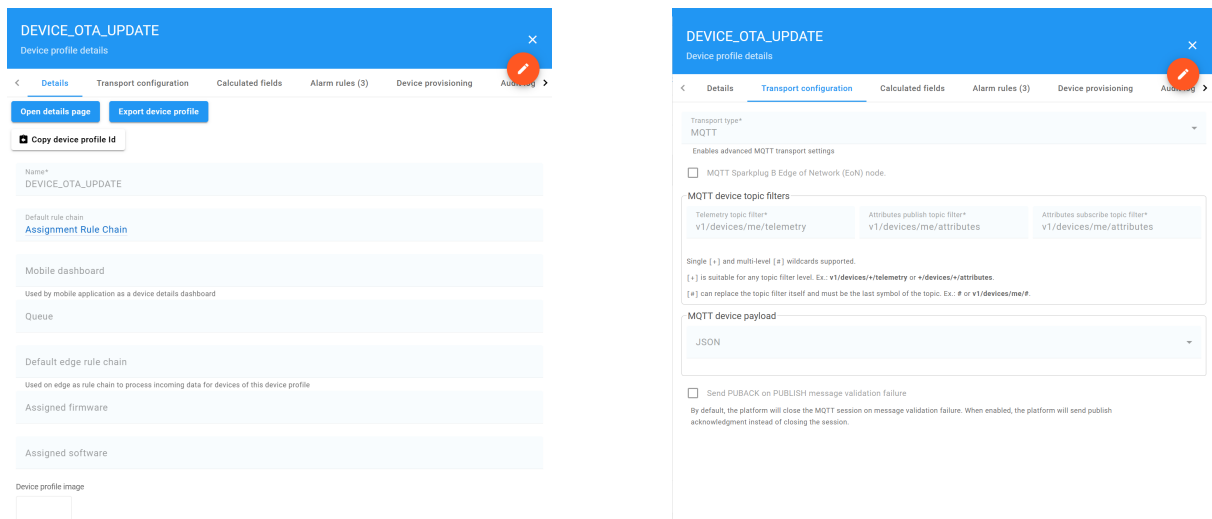


Figure 4.5: Device profile configuration: (left) assigned rule chain, (right) MQTT transport setup.

In this example, the profile `DEVICE_OTA_UPDATE` uses the `Assignment Rule Chain` as its default. This ensures all telemetry, RPCs, and attribute updates from devices using this profile are passed through a standardized flow.

4.3.2 Alarm Rules Configuration

The system implements a unified alarm mechanism by separating rule logic and threshold values. Rules are defined centrally in the device profile, while thresholds are configured per device and dynamically referenced during evaluation. This design improves scalability and customization.

Each device instance (e.g., `ESP32_OTA`) is assigned the profile `DEVICE_OTA_UPDATE`. Device-level alarm thresholds are manually configured via attributes and include:

- **High Temperature Threshold:** 20°C
- **Low Humidity Threshold:** 100%
- **Low Light Intensity Threshold:** 20 lx

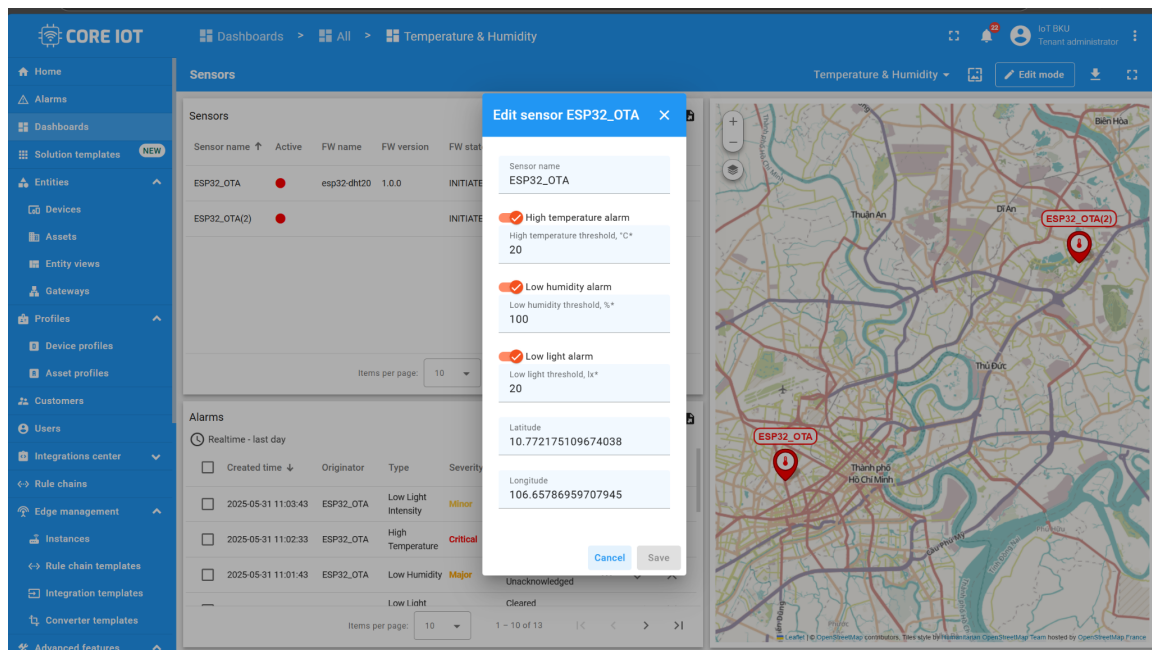


Figure 4.6: Configuring alarm thresholds for temperature, humidity, and light directly on the device.

The alarm logic is defined in the device profile and references the device's threshold attributes. This allows one rule to apply to multiple devices with different threshold values.

High Temperature Alarm (Critical)

- **Trigger:** When `temperatureAlarmFlag = True` and `temperature > temperatureAlarmThreshold`
- **Clear:** When `temperature <= temperatureAlarmThreshold`

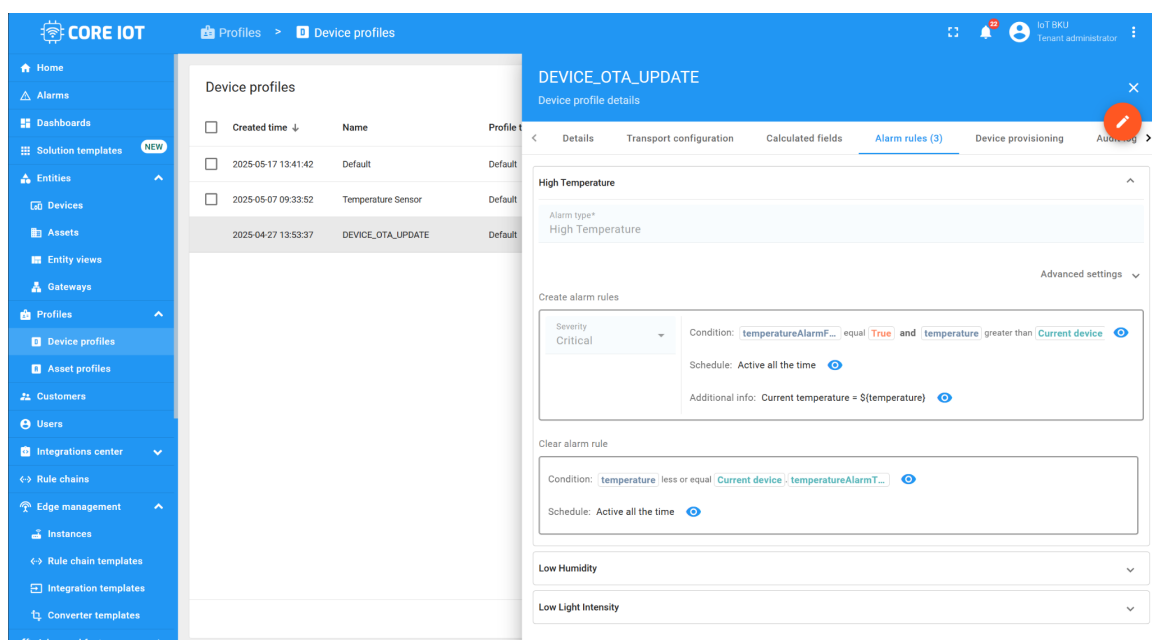


Figure 4.7: High temperature alarm rule logic in device profile referencing the device's threshold attribute.

Low Humidity Alarm (Major)

- **Trigger:** When `humidityAlarmFlag = True` and `humidity < humidityAlarmThreshold`
- **Clear:** When `humidity > humidityAlarmThreshold`

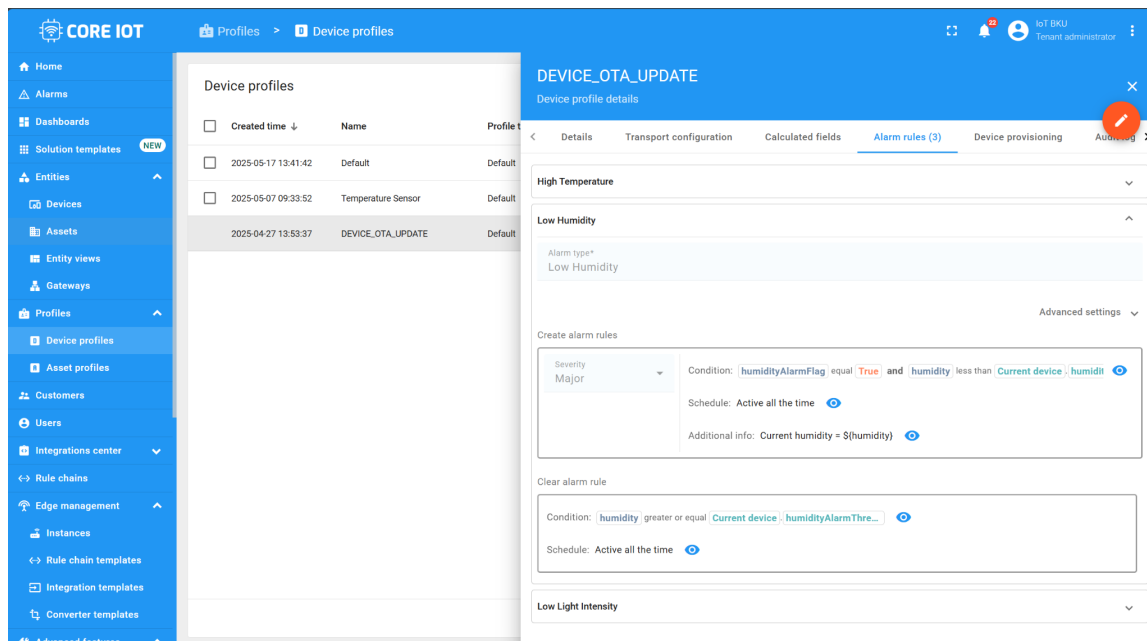


Figure 4.8: Low humidity alarm rule with dynamic threshold referencing per device.

Low Light Intensity Alarm (Minor)

- **Trigger:** When `lightAlarmFlag = True` and `light < lightAlarmThreshold`
- **Clear:** When `light > lightAlarmThreshold`

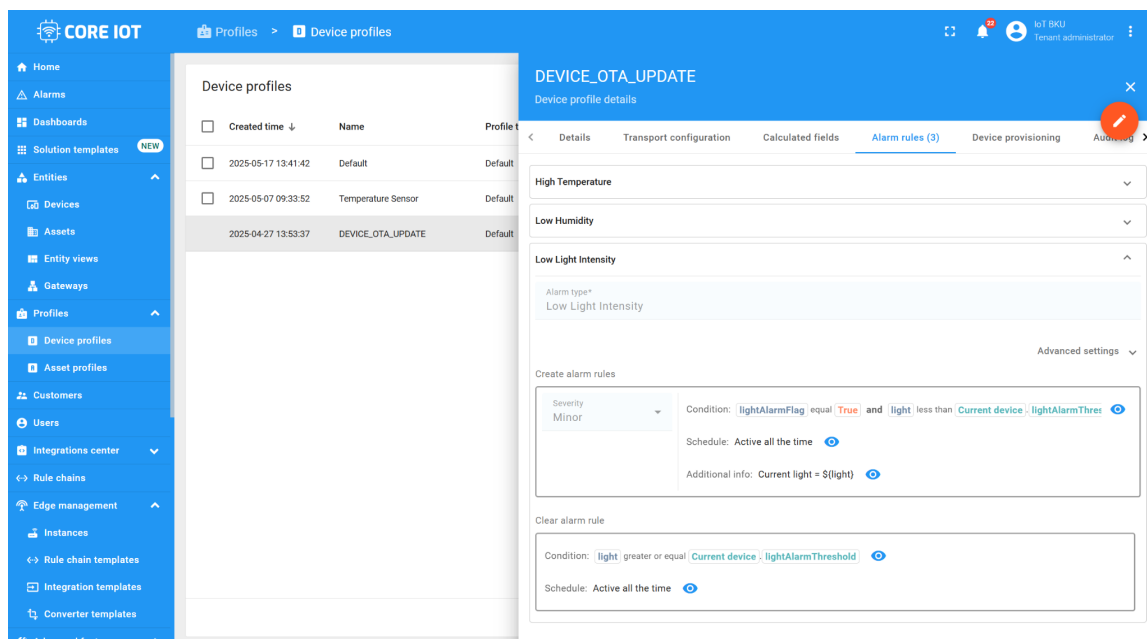


Figure 4.9: Low light intensity alarm rule with minor severity in the device profile.

This approach ensures consistency, reduces redundancy in rule configuration, and allows operators to easily customize sensitivity on a per-device basis.

4.3.3 Rule Chain Execution

In the Core IoT platform, rule chains serve as the backbone of data processing workflows. Each rule chain is a directed graph of logic nodes that interpret incoming messages from devices (e.g., telemetry, attribute updates, RPC requests), apply filters, and trigger actions like alarms, RPC calls, or data persistence.

In this system, the device profile `DEVICE_OTA_UPDATE` uses the Assignment Rule Chain as its default rule chain. The rule chain visually depicts the logical flow for all incoming messages related to the device, including telemetry, RPC, and attribute updates.

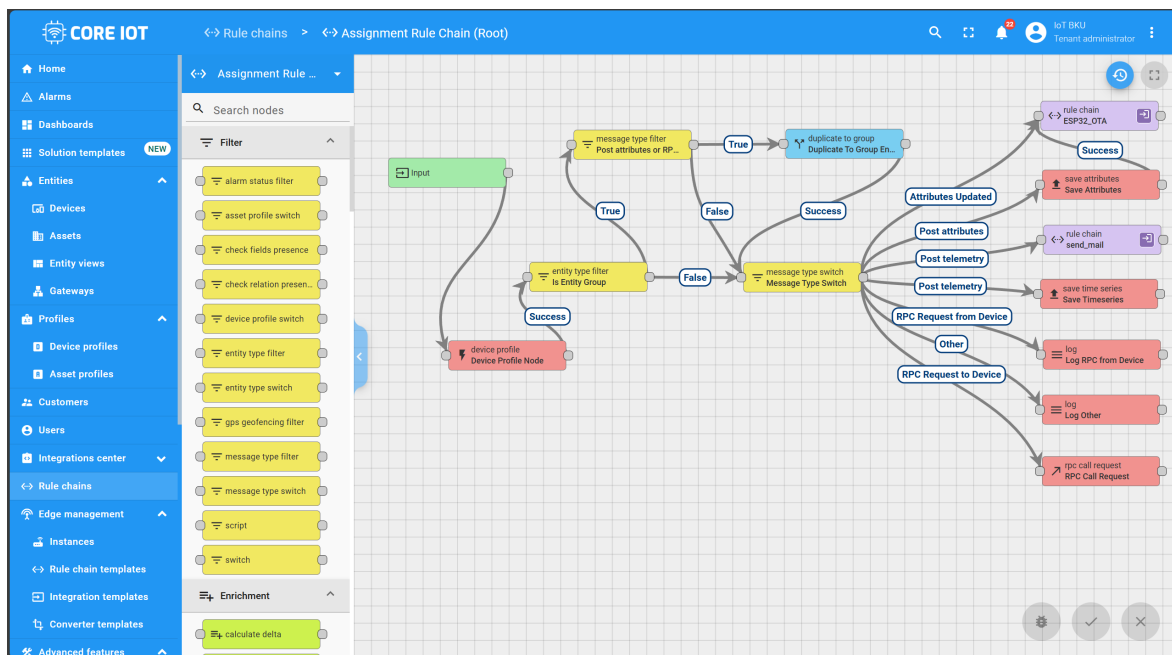


Figure 4.10: Assignment Rule Chain logic: handles incoming messages and directs them through type-specific filters and actions.

The logic follows these key stages:

- **Input Node:** Entry point for all device-originated data packets (telemetry, RPC, attributes, etc.).
- **Message Type Filter:** Determines whether the message is a telemetry post, RPC request, or attribute update. Each path leads to distinct handling logic.
- **Device Profile Node:** Ensures that the logic aligns with the attached device profile. This validates and retrieves metadata for later use.
- **Entity Type Filter:** Filters whether the entity belongs to a specific group or type, enabling selective routing or multi-tenant separation if needed.

- **Message Type Switch:** This node classifies the message into subtypes such as:
 - `Post attributes` → Attributes are saved using the `Save Attributes` node.
 - `Post telemetry` → Data is persisted using the `Save Timeseries` node.
 - `RPC Request from Device` → Logs the event using `Log RPC from Device`.
 - `RPC Request to Device` → Executes RPC using `RPC Call Request`.
- **Rule Chain Forwarding:** Some paths (e.g., `Attributes Updated`) forward data to child rule chains like `ESP32_OTA` or `send_mail` for additional workflows such as notification or OTA handling.
- **Logging and Debugging Nodes:** `Log RPC from Device` and `Log Other` nodes help in diagnostics, capturing unhandled message types or RPC requests for audit or debugging purposes.

This rule chain structure is both modular and extensible, allowing future enhancements like threshold checking, AI processing, or external API triggering to be easily inserted into the pipeline. It also ensures telemetry is correctly routed for visualization and alerts.

4.4 Web Application Integration

To enhance accessibility and visualization, a custom-built web application is integrated with the Core IoT platform to display real-time telemetry. The frontend is developed using React and connects to Core IoT via its REST API and WebSocket for telemetry data retrieval.

The web interface includes a homepage displaying all connected devices with location mapping and device summaries.

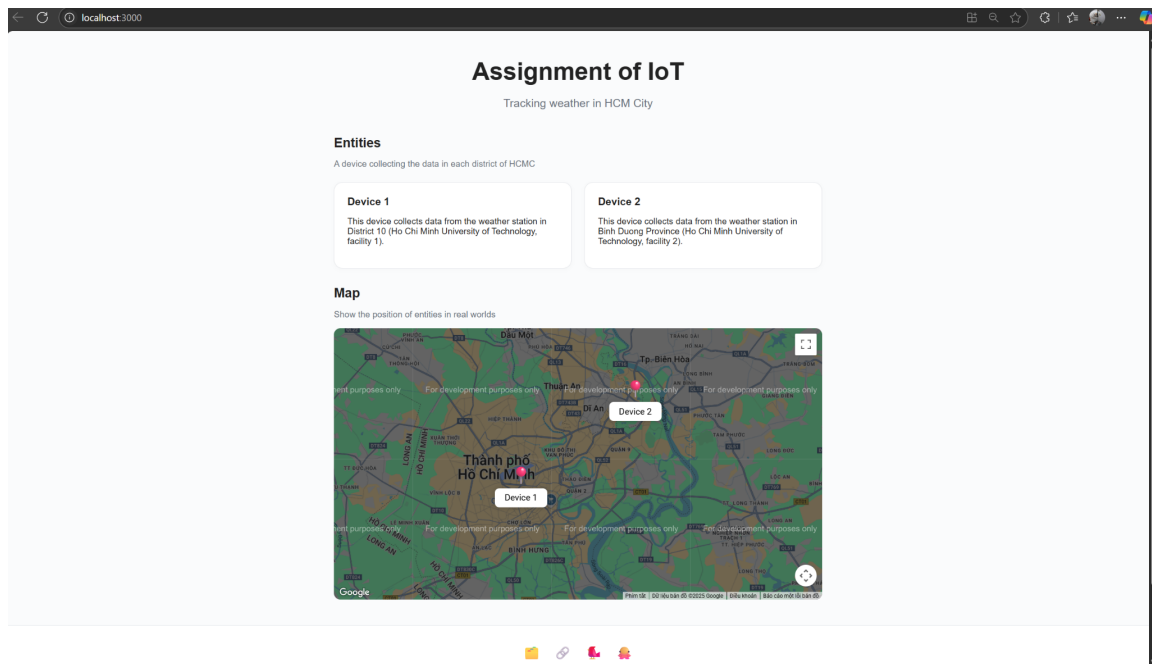


Figure 4.11: Homepage of the Web Application displaying registered devices and their locations in HCMC and Binh Duong.

Clicking on a specific device navigates the user to a detailed view page. This view features real-time charts for environmental data such as temperature, humidity, and light intensity. These metrics are updated continuously using telemetry pushed from Core IoT.

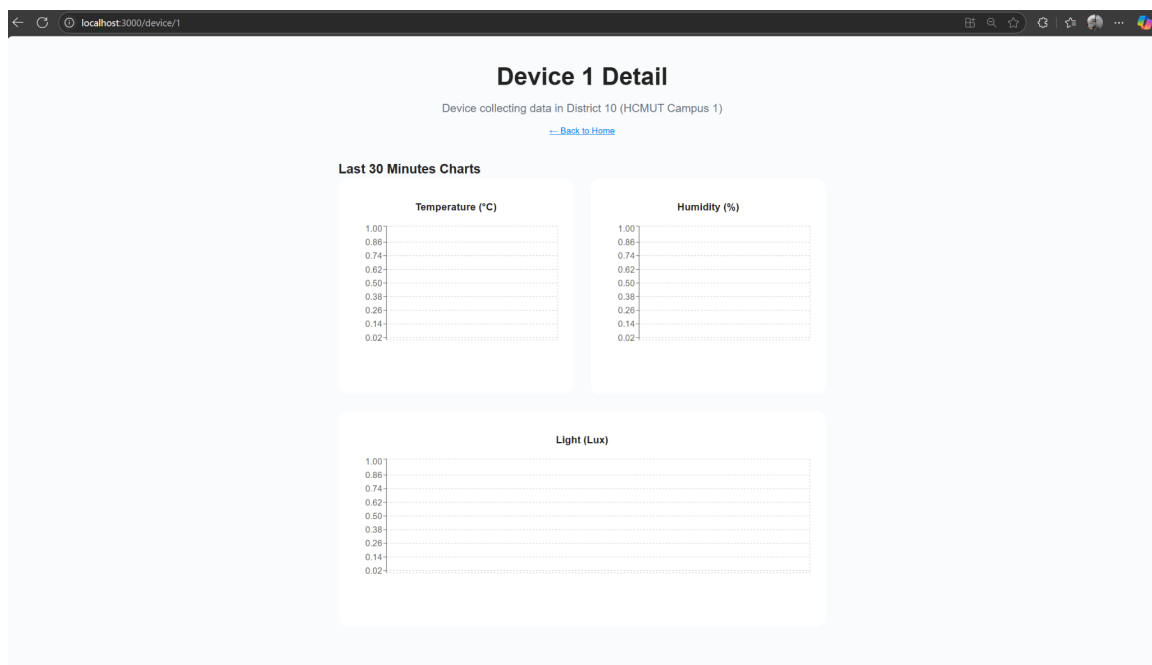


Figure 4.12: Device Detail View with live telemetry charts for Temperature, Humidity, and Light.

The web application provides users with:

- **Entity Summary:** Lists all devices, location, and status.

- **Live Charts:** Fetches and plots telemetry data in real-time for up-to-date monitoring.
- **Map View:** Uses embedded Google Maps to visualize sensor positions.
- **Responsive UI:** Mobile-optimized and user-friendly navigation.

This integration demonstrates how real-time data from edge devices and sensors can be visualized in an accessible and scalable web environment.

4.5 Demo Link

The demo of our system can be access at:

- Demo of our system: ([LINK](#))
- GitHub repository: ([LINK](#))

CHAPTER 5

Prototype: Over-the-air Update

Over-the-air (OTA) update is a critical feature in modern IoT systems, enabling remote firmware and software updates without requiring physical access to the device. This section presents a prototype implementation of the OTA update mechanism for the Yolo UNO board, integrated within the Core IoT platform. The prototype demonstrates how firmware can be delivered, verified, and installed over a network connection, ensuring minimal disruption and maximum device uptime. The system leverages secure channels and digital signatures to maintain update integrity and prevent unauthorized modifications. This approach significantly enhances maintainability, scalability, and lifecycle management of deployed IoT devices in the field.

Why it is a prototype: Due to limitations in time and technical knowledge, the microcontroller unit (MCU) was connected directly to the Core IoT platform, bypassing a true edge-device architecture. In a proper IoT system design, only the edge device should communicate with the cloud platform, acting as a gateway for lower-level hardware like the MCU. Therefore, this implementation is considered a prototype, intended to demonstrate that the Core IoT platform is capable of performing over-the-air (OTA) updates. The demonstration follows the standard OTA update workflow to validate functionality despite the simplified architecture.

5.1 The Implementation and Results of OTA

5.1.1 Designing the Rule Chain for OTA Updates

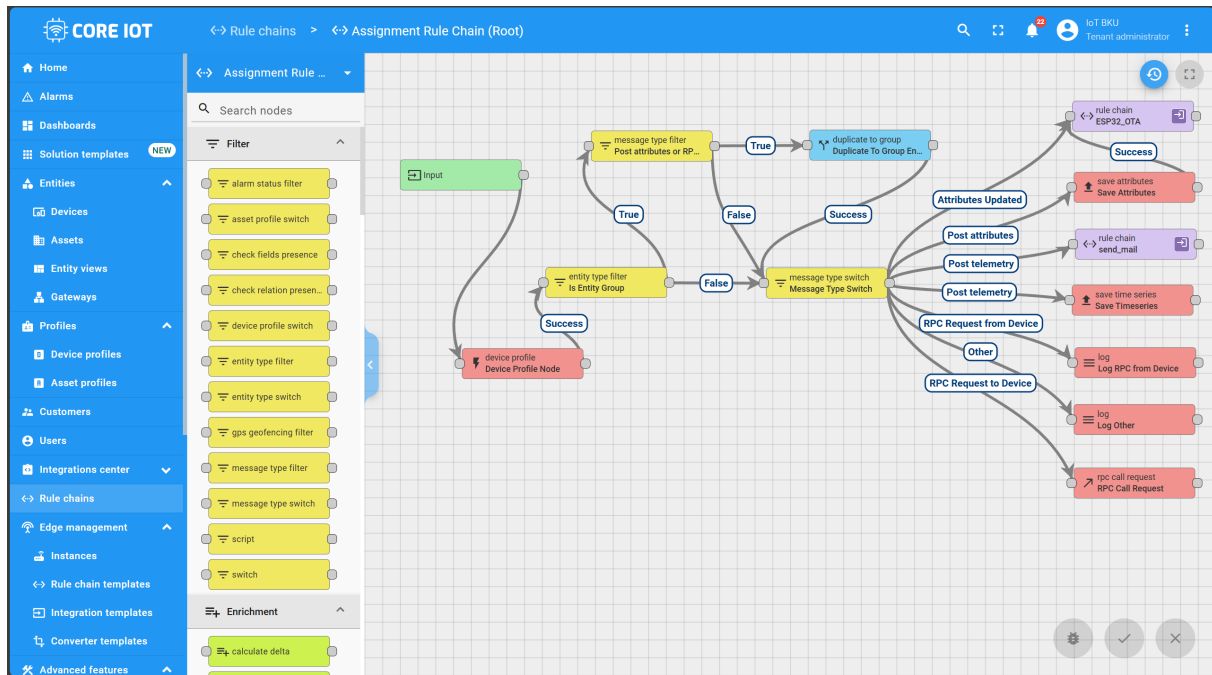


Figure 5.1: Design main rule chain for configuration OTA Update

The main rule chain serves as the central logic processor for handling incoming messages from IoT devices. As illustrated in Figure 5.1, it begins with an Input node that receives all data from connected devices. Messages are first passed through filters based on type and entity group status to determine appropriate routing.

If the message is of type Post attributes or RPC, it is duplicated to the appropriate group using the Duplicate To Group Entity node. All other messages are evaluated through a Message Type Switch, which directs them to corresponding branches such as telemetry storage, logging, sending mail, or invoking a sub-rule chain like ESP32_OTA.

This architecture ensures modular processing, enabling OTA logic to be isolated in a dedicated sub-chain while maintaining a scalable and maintainable message-handling system.

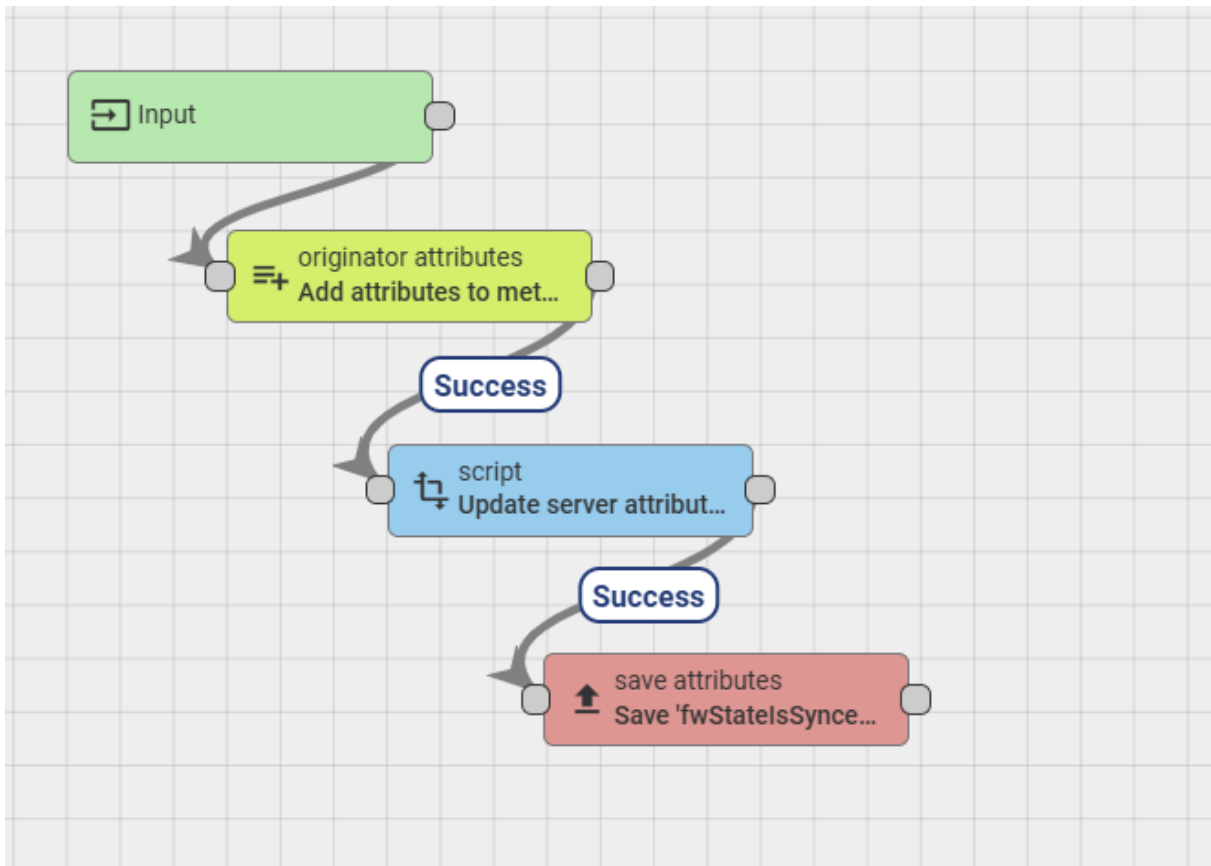


Figure 5.2: Sub rule chain for configuration of OTA update

The ESP32_OTA rule chain is a dedicated subroutine responsible for managing OTA (Over-the-Air) update operations. As shown in Figure 5.2, it is triggered when the platform detects an update-related event, such as a change in firmware version attributes.

The rule chain begins with an Input node, followed by a step to append additional metadata using the Add attributes to metadata node. A script node, Update server attributes, processes the firmware version data and determines whether a firmware update is necessary. If the update is valid, the final node Save 'fwStateIsSynced' commits the status change to the device attributes.

This rule chain isolates OTA logic from the main flow, improving reusability and reducing coupling in system design. It also provides clear logging and state tracking of firmware synchronization.

5.1.2 Flow chart for integrating the ESP32 with the Core IoT SDK

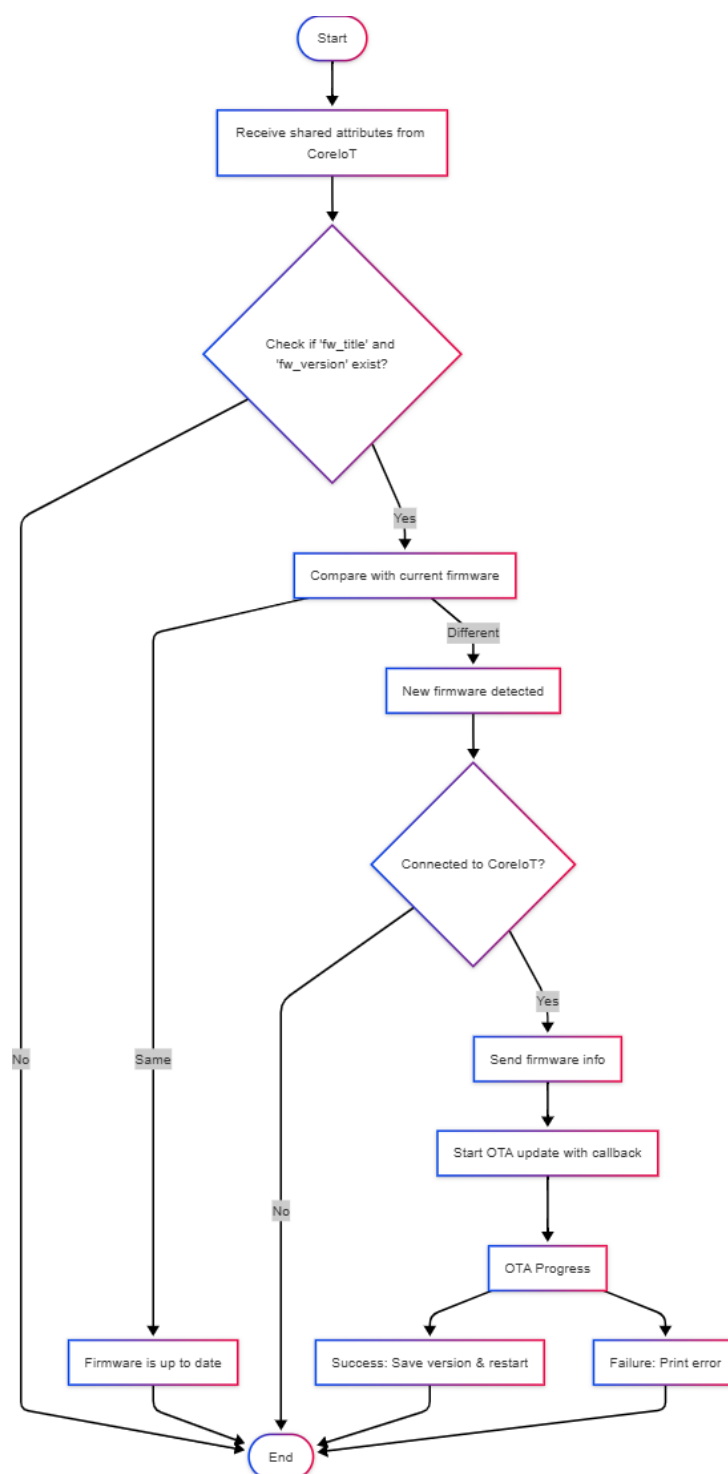


Figure 5.3: Flowchart of OTA update in MCU

5.1.3 Uploading and Assigning Firmware on the Core IoT Platform

5.1.3.1 Uploading Firmware in Core IoT Platform

To enable Over-the-Air (OTA) updates on devices via the Core IoT platform, firmware packages must first be uploaded and managed within the OTA update system.

Figure 5.7 illustrates the interface for adding a new firmware package. The steps to upload firmware are as follows:

1. Navigate to **Advanced features** → **OTA updates** from the main menu.
2. Click the “+” icon to open the *Add package* dialog.
3. Fill in the required fields:
 - **Title:** A name for the firmware package (e.g., esp32-dht20).
 - **Version:** Firmware version (e.g., 1.0.2).
 - **Version tag:** A string that matches the version reported by the device.
 - **Device profile:** Select the appropriate device profile (e.g., ESP32).
 - **Package type:** Choose Firmware.
4. Under **Package file**, select Upload binary file and upload the compiled .bin firmware.
5. Optionally check Auto-generate checksum for file verification.
6. Click **Add** to complete the upload.

Once uploaded, the firmware will appear in the package list and can be assigned to target devices for OTA deployment.

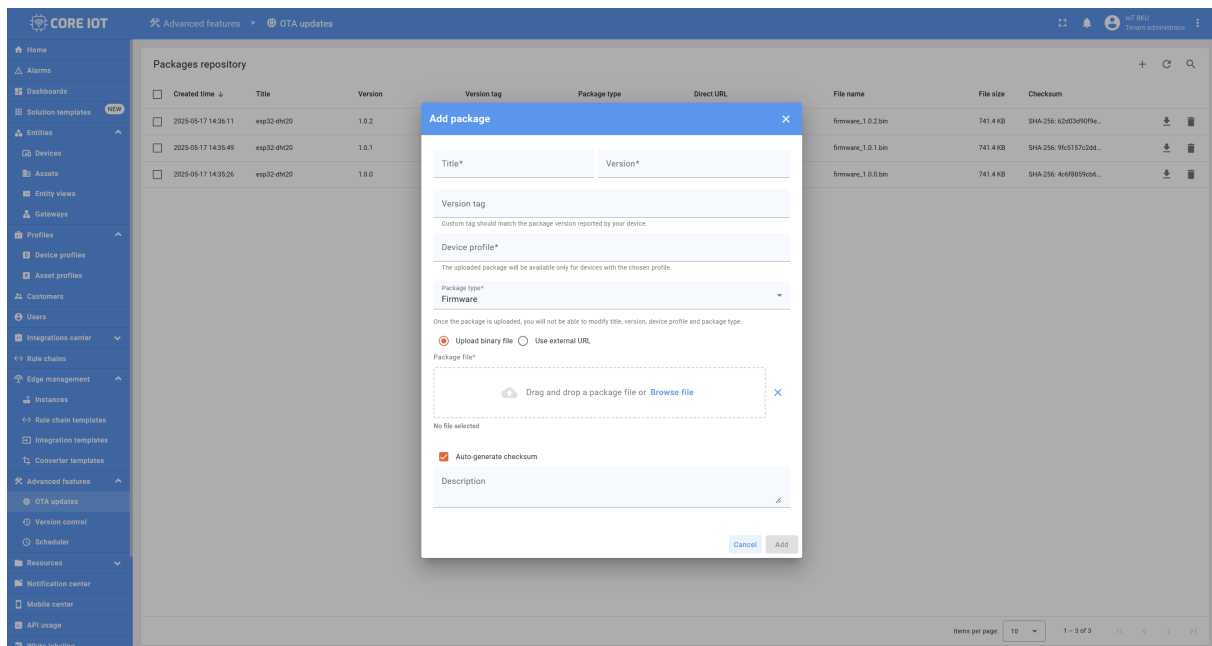


Figure 5.4: Upload firmware in Core IoT Platform

5.1.3.1.a Uploading Firmware in Core IoT Platform

To enable Over-the-Air (OTA) updates on devices via the Core IoT platform, firmware packages must first be uploaded and managed within the OTA update system.

Figure 5.7 illustrates the interface for adding a new firmware package. The steps to upload firmware are as follows:

1. Navigate to **Advanced features** → **OTA updates** from the main menu.
2. Click the up-arrow icon to open the *Add package* dialog.
3. Fill in the required fields:
 - **Title:** A name for the firmware package (e.g., esp32-dht20).
 - **Version:** Firmware version (e.g., 1.0.2).
4. Click **Save** to complete the upload.

Once uploaded, the firmware will appear in the package list and can be assigned to target devices for OTA deployment.

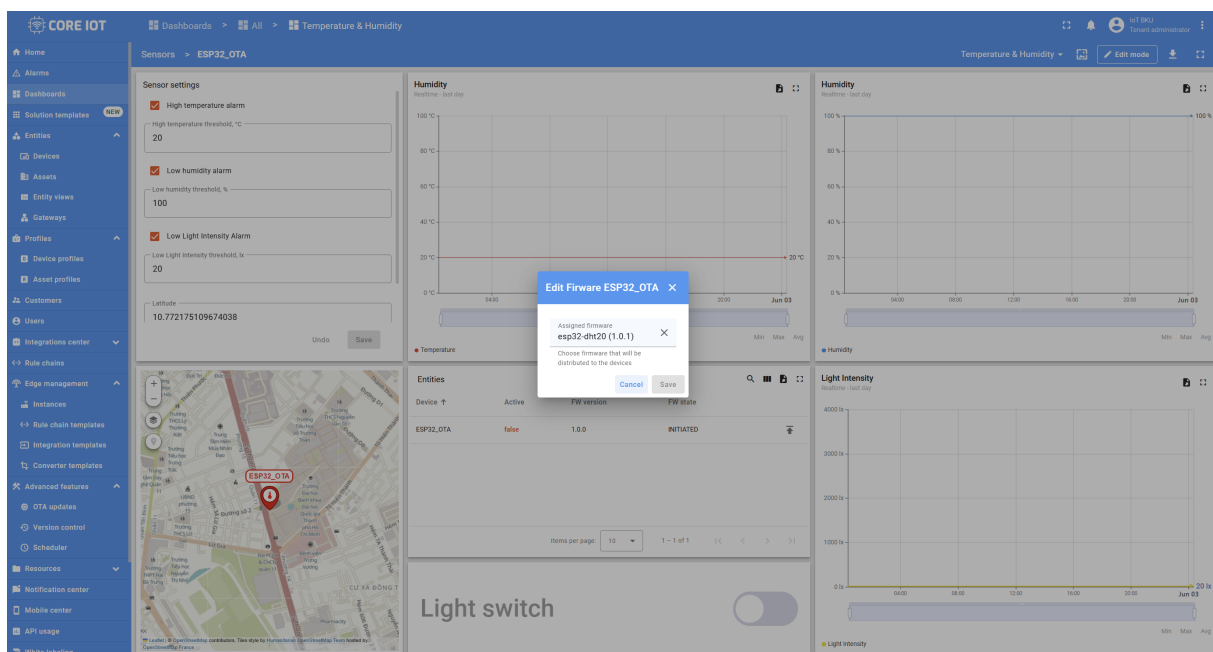
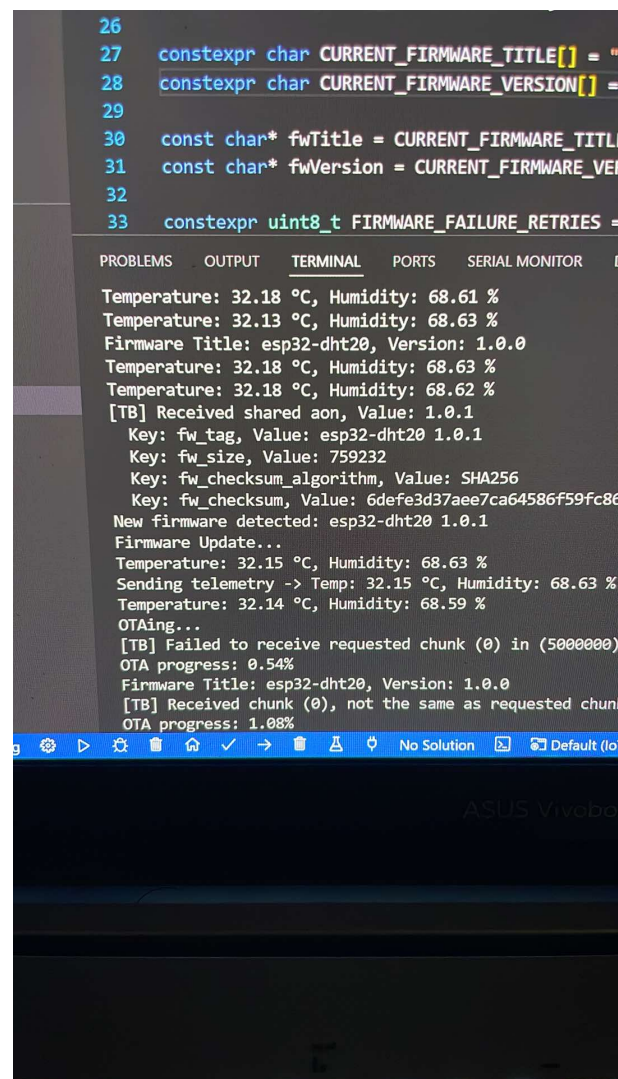


Figure 5.5: Firmware Upload Interface in Core IoT Platform

5.1.4 OTA Update Results and Verification

```
Firmware Title: esp32-dht20, Version: 1.0.0
DATA:28.80, 50.82, 0
Firmware Title: esp32-dht20, Version: 1.0.0
DATA:28.79,Firmware Title: esp32-dht20, Version: 1.0.0
```

Figure 5.6: The firmware version before updating



```

26
27 constexpr char CURRENT_FIRMWARE_TITLE[] = "
28 constexpr char CURRENT_FIRMWARE_VERSION[] =
29
30 const char* fwTitle = CURRENT_FIRMWARE_TITL
31 const char* fwVersion = CURRENT_FIRMWARE_VE
32
33 constexpr uint8_t FIRMWARE_FAILURE_RETRIES =

```

PROBLEMS OUTPUT **TERMINAL** PORTS SERIAL MONITOR

```

Temperature: 32.18 °C, Humidity: 68.61 %
Temperature: 32.13 °C, Humidity: 68.63 %
Firmware Title: esp32-dht20, Version: 1.0.0
Temperature: 32.18 °C, Humidity: 68.63 %
Temperature: 32.18 °C, Humidity: 68.62 %
[TB] Received shared aon, Value: 1.0.1
Key: fw_tag, Value: esp32-dht20 1.0.1
Key: fw_size, Value: 759232
Key: fw_checksum_algorithm, Value: SHA256
Key: fw_checksum, Value: 6defe3d37aee7ca64586f59fc86
New firmware detected: esp32-dht20 1.0.1
Firmware Update...
Temperature: 32.15 °C, Humidity: 68.63 %
Sending telemetry -> Temp: 32.15 °C, Humidity: 68.63 %
Temperature: 32.14 °C, Humidity: 68.59 %
OTAing...
[TB] Failed to receive requested chunk (0) in (5000000)
OTA progress: 0.54%
Firmware Title: esp32-dht20, Version: 1.0.0
[TB] Received chunk (0), not the same as requested chunk
OTA progress: 1.08%

```

ASUS VivoBook

Figure 5.7: Firmware 1.0.1 Update after assignng

CHAPTER 6

CONCLUSION

6.1 Reflections and Learned Knowledge

Through the completion of this project, we have gained a deep and practical understanding of core concepts in the field of Internet of Things (IoT), embedded systems, edge computing, and artificial intelligence. Specifically, we:

- Learned to integrate microcontrollers (ESP32) with various environmental sensors and program them using FreeRTOS and MicroPython.
- Understood the architecture and implementation of serial communication between hardware and software layers.
- Gained experience in building a full edge-to-cloud pipeline, including edge-based AI inference and MQTT-based cloud communication.
- Mastered the design and customization of dashboards using Core IoT for real-time telemetry visualization, alarm configuration, and rule chain execution.
- Developed a scalable and interactive web application using modern frontend technologies (React, TypeScript, Google Maps API) that connects to a backend through REST and WebSocket APIs.
- Explored practical applications of anomaly detection using CNN-based time-series forecasting and integrated it with real-world alert systems.

6.2 Project Achievements

Building upon the above knowledge, this project successfully delivered a modular and extensible IoT monitoring system that:

- Collects environmental data from ESP32-based sensor nodes and transfers it to a local edge device via serial communication.
- Utilizes an AI model on the edge to predict future values and detect anomalies in temperature and humidity readings.
- Sends telemetry and prediction data to a cloud platform (Core IoT), where it is visualized in real-time on an interactive dashboard.
- Implements a rule-based system that automatically sends Web alerts on Core IoT platform to notify users when environmental anomalies are detected.
- Provides a user-facing web application that displays device locations, live charts, and device details with secure login and responsive design.

These outcomes demonstrate the system's practical value and its readiness for use in real-time monitoring scenarios such as smart buildings, agriculture, or remote lab environments.

6.3 Limitations of the Project

Despite its success, the project has a few limitations:

- **Fixed Sensor Types:** The current system only supports DHT20 (temperature/humidity) and a light sensor. Adding new sensor types would require firmware updates and front-end/backend integration.
- **Offline Training Requirement:** The AI model used for prediction and anomaly detection must be trained offline and cannot currently adapt to new patterns in real time.
- **Manual Threshold Configuration:** Alarm thresholds must be configured manually per device, which can become tedious in large-scale deployments.
- **Limited AI Complexity:** The AI model is relatively lightweight (CNN with shallow layers), which may not generalize well to more complex sensor dynamics or environmental variability.
- **Local Dependence of Edge Device:** The edge application is run on a laptop, which may not be ideal for long-term or mobile deployments unless replaced with a more ruggedized or embedded edge computing device.
- **OTA update on Edge Device:** OTA shall be handled by edge device but our project is still not implemented

6.4 Future Extensions and Vision

To build upon the current achievements, the project can be extended in the following directions:

- **Scalable Edge Platform:** Deploy the edge software on low-cost, power-efficient edge devices such as Raspberry Pi or NVIDIA Jetson to allow for more permanent or mobile setups.
- **Online Learning and Adaptive AI:** Implement an online learning mechanism that updates the anomaly detection model dynamically based on newly collected data, improving long-term accuracy.
- **Sensor Abstraction Layer:** Develop a plugin-like system where new sensor types can be easily integrated without modifying the core system logic.
- **Advanced Analytics Dashboard:** Introduce more AI-based features such as pattern recognition, forecasting trends, or energy efficiency recommendations using historical data.
- **Mobile App Development:** Expand the user access channel with a mobile-native app that offers real-time push notifications, device management, and offline capabilities.

Long-term Vision: If further developed, this system can be transformed into a commercial-ready Smart Environment Monitoring Platform capable of being deployed in:

- Smart campuses and schools for indoor environmental quality tracking.
- Agricultural farms to monitor growing conditions and detect irrigation needs.
- Manufacturing facilities for early detection of overheating or humidity-sensitive equipment failures.
- Smart city applications to create distributed sensor networks for air quality, noise, and climate monitoring.

With continued research and integration of 5G, federated learning, and predictive automation, the system has the potential to become part of a broader AIoT ecosystem that improves quality of life, enhances sustainability, and powers next-generation smart infrastructure.

CHAPTER 7

REFERENCES

1. Demo of our system: ([LINK](#))
2. GitHub repository: ([LINK](#))